

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka
Specjalność: Inżynieria Systemów Informatycznych

PRACA DYPLOMOWA
MAGISTERSKA

**Projekt i analiza porównawcza wydajności skalowalnych architektur
systemów typu SaaS**

OSABOSA

Opiekun pracy
dr hab. inż. Imię Nazwisko

Słowa kluczowe: skalowalność, Kubernetes, HPA, mikroserwisy, SaaS, testy obciążeniowe,
Prometheus, autoskalowanie

Spis treści

1	Wstęp teoretyczny	1
1.1	Motywacja i kontekst badawczy	1
1.2	Problem badawczy i hipoteza	1
1.3	Cel i zakres pracy	1
1.4	Architektury skalowalne systemów	2
1.4.1	Monolit, SOA i mikroserwisy	2
1.4.2	Wzorce skalowania	2
1.5	Platformy orkiestracji kontenerów	3
1.5.1	Kubernetes — architektura i kluczowe komponenty	3
1.5.2	k3s — lekka dystrybucja Kubernetes	3
1.6	Mechanizmy autoskalowania w Kubernetes	3
1.6.1	HorizontalPodAutoscaler (HPA)	3
1.6.2	VerticalPodAutoscaler (VPA)	4
1.6.3	KEDA	4
1.7	Modele SaaS i multi-tenancy	4
1.7.1	Software as a Service (SaaS)	4
1.7.2	Multi-tenancy	4
1.8	Przegląd literatury i istniejących badań	5
1.8.1	Badania nad HPA w Kubernetes	5
1.8.2	Luki badawcze	5
1.9	Struktura pracy	5
2	Przegląd technologii	7
2.1	Kubernetes jako platforma orkiestracji	7
2.1.1	Architektura	7
2.1.2	Kluczowe obiekty API	7
2.1.3	HorizontalPodAutoscaler (HPA) — szczegółowo	8
2.2	k3s — lekka dystrybucja Kubernetes	8
2.2.1	Charakterystyka	8
2.2.2	Różnice względem standardowego Kubernetes	8
2.2.3	Uzasadnienie wyboru	8
2.3	Prometheus i Grafana — stos monitorujący	9
2.3.1	Prometheus	9
2.3.2	Grafana	9
2.3.3	Prometheus Adapter	10
2.4	k6 — narzędzie do testów obciążeniowych	10
2.4.1	Charakterystyka	10
2.4.2	Scenariusze k6 w projekcie	11
2.5	Docker i konteneryzacja	11

2.5.1	Docker	11
2.5.2	Docker Compose	11
2.6	Traefik jako Ingress Controller	11
2.7	GKE (Google Kubernetes Engine) — wdrożenie chmurowe	12
2.7.1	Charakterystyka GKE	12
2.7.2	Porównanie k3s vs GKE	12
2.8	Podsumowanie stosu technologicznego	13
3	Projekt architektury systemu badawczego	15
3.1	Macierz badawcza	15
3.2	Aplikacje badawcze	16
3.2.1	CPU-service — obciążenie CPU-bound	16
3.2.2	IO-service — obciążenie I/O-bound	16
3.2.3	Echo-server — serwis pomocniczy	16
3.2.4	Memory-service — serwis pamięciowy (dodatkowy)	17
3.3	Strategie HPA	17
3.3.1	HPA CPU (Resource Metric)	17
3.3.2	HPA Memory (Resource Metric)	17
3.3.3	HPA Custom RPS (Pods Metric)	18
3.4	Infrastruktura monitorująca	18
3.4.1	Prometheus	18
3.4.2	Grafana	18
3.4.3	Prometheus Adapter	19
3.5	Topologia wdrożeniowa	19
3.5.1	Zasoby przydzielone serwisom	20
3.6	Plan eksperymentów	20
3.6.1	Scenariusze testowe	20
3.6.2	Procedura eksperymentalna	20
3.6.3	Profile obciążeniowe k6	21
3.6.4	Metryki badawcze	21
3.7	Podsumowanie architektury	22
4	Implementacja	23
4.1	Struktura repozytorium	23
4.2	Implementacja mikroservisów	24
4.2.1	Wspólne cechy implementacyjne	24
4.2.2	CPU-service	24
4.2.3	IO-service	25
4.2.4	Echo-server	28
4.2.5	Memory-service	28
4.3	Konfiguracja Kubernetes	28
4.3.1	Namespace i struktura zasobów	28
4.3.2	Deploymenty	28
4.3.3	HPA	30
4.3.4	Services	31
4.3.5	Ingress	31
4.4	Infrastruktura monitorująca	32
4.4.1	Prometheus	32
4.4.2	Grafana	32

4.4.3	Prometheus Adapter	32
4.5	Skrypty automatyzujące	34
4.5.1	deploy-k3s.sh — wdrożenie infrastruktury	34
4.5.2	run-tests.sh — orkiestracja testów	34
4.5.3	run-all.sh — pełny pipeline	36
4.6	Scenariusze testowe k6	36
4.6.1	cpu-load-test.js	36
4.6.2	io-load-test.js	37
4.6.3	combined-scenario.js	38
4.7	Podsumowanie implementacji	39
5	Badania eksperymentalne i analiza wyników	41
5.1	Metodyka badań	41
5.1.1	Środowisko testowe	41
5.1.2	Procedura eksperymentalna	41
5.1.3	Zbierane metryki	42
5.1.4	Metryki badawcze — definicje operacyjne	42
5.2	Scenariusze testowe	43
5.2.1	Scenariusze bazowe (bez HPA)	43
5.2.2	Scenariusze CPU-bound	43
5.2.3	Scenariusze I/O-bound	43
5.3	Wyniki eksperymentalne — środowisko lokalne (k3s)	44
5.3.1	Scenariusze bazowe	44
5.3.2	Scenariusze CPU-bound — porównanie strategii HPA	44
5.3.3	Scenariusze I/O-bound — porównanie strategii HPA	45
5.3.4	Wykresy przebiegu skalowania	45
5.4	Analiza wyników	45
5.4.1	Porównanie strategii HPA dla CPU-bound	45
5.4.2	Porównanie strategii HPA dla I/O-bound	45
5.4.3	Efektywność skalowania	46
5.5	Wyniki eksperymentalne — środowisko chmurowe (GKE)	46
5.6	Dyskusja i interpretacja	46
5.7	Wnioski z badań	46
6	Rozszerzenie <i>SaaS Multi-Tenancy</i>	47
6.1	Status implementacji	47
6.2	Koncepcja <i>Multi-Tenancy</i> w systemie badawczym	47
6.2.1	Model izolacji tenantów	47
6.2.2	Projektowana architektura	47
6.3	Planowane scenariusze testowe	48
6.4	Oczekiwane wyzwania	48
6.5	Podsumowanie	49
7	Podsumowanie	51
7.1	Podsumowanie zrealizowanych prac	51
7.2	Weryfikacja hipotezy badawczej	51
7.3	Ograniczenia pracy	52
7.4	Rekomendacje praktyczne	53
7.5	Dalsze kierunki badań	53

7.6	Podsumowanie końcowe	54
A	Spis listingów	55

Rozdział 1

Wstęp teoretyczny

1.1 Motywacja i kontekst badawczy

Współczesne systemy informatyczne funkcjonują w środowisku charakteryzującym się zmiennym obciążeniem, rosnącymi oczekiwaniami użytkowników dotyczącymi czasu odpowiedzi oraz koniecznością efektywnego wykorzystania zasobów sprzętowych. Model *SaaS* (*Software as a Service*) narzuca dodatkowe wymagania związane z obsługą wielu klientów (*tenantów*) współdzielących tę samą infrastrukturę przy zachowaniu izolacji danych i przewidywalnej wydajności.

Kluczowym mechanizmem umożliwiającym sprostanie tym wymaganiom jest automatyczne skalowanie (*autoscaling*) — zdolność systemu do dynamicznego dostosowywania liczby jednostek obliczeniowych do bieżącego obciążenia bez interwencji operatora. W ekosystemie *Kubernetes* podstawowym narzędziem realizującym skalowanie poziome jest *HorizontalPodAutoscaler* (HPA), który na podstawie obserwowanych metryk podejmuje decyzje o dodaniu lub usunięciu replik aplikacji.

1.2 Problem badawczy i hipoteza

Skuteczność mechanizmu HPA zależy od wyboru metryki, na podstawie której podejmowana jest decyzja o skalowaniu. Metryka musi być skorelowana z rzeczywistym obciążeniem aplikacji — w przeciwnym razie system może nie skalować się w ogóle (niedoskalowanie, *underscaling*) lub skalować się nadmiernie (przeskalowanie, *overscaling*). Problem ten jest szczególnie istotny w architekturach *SaaS*, gdzie różni tenantci generują obciążenia o odmiennych charakterystykach.

Hipoteza badawcza: Nie istnieje jedna uniwersalna metryka HPA — skuteczność strategii skalowania zależy od charakterystyki obciążeniowej aplikacji (*workload profile*). Błędny wybór metryki prowadzi do niedoskalowania lub przeskalowania.

1.3 Cel i zakres pracy

Celem niniejszej pracy jest:

1. Zaprojektowanie i implementacja prototypu skalowalnego systemu w modelu *SaaS*, składającego się z mikroservisów o diametralnie różnych profilach obciążeniowych (*CPU-bound* oraz *I/O-bound*).

2. Przeprowadzenie kontrolowanych eksperymentów porównujących trzy strategie HPA: opartą na zużyciu CPU, opartą na zużyciu pamięci oraz opartą na niestandardowej metryce RPS (*Requests Per Second*) dostarczanej przez *prometheus-adapter*.
3. Analiza porównawcza efektywności strategii skalowania w macierzy badawczej 2×3 (dwa typy obciążenia $\times$ trzy strategie HPA).
4. Sformułowanie rekomendacji dotyczących doboru metryki HPA w zależności od charakterystyki aplikacji.

1.4 Architektury skalowalne systemów

1.4.1 Monolit, SOA i mikroserwisy

Ewolucja architektur systemów rozproszonych przebiegała od monolitycznych aplikacji, przez architekturę zorientowaną na usługi (*SOA*), do współczesnego paradygmatu mikroserwisów. Każde z tych podejść oferuje inny kompromis między złożonością zarządzania a elastycznością skalowania:

- **Monolit** — cała funkcjonalność systemu zawarta w jednej jednostce wdrożeniowej. Skalowanie odbywa się przez uruchomienie wielu instancji całej aplikacji, co jest nieefektywne, gdy tylko wybrane komponenty wymagają większej mocy obliczeniowej.
- **SOA** (*Service-Oriented Architecture*) — system podzielony na usługi komunikujące się przez *Enterprise Service Bus* (ESB). Umożliwia selektywne skalowanie, ale wprowadza złożoność infrastrukturalną i potencjalne wąskie gardła w warstwie integracyjnej.
- **Mikroserwisy** — każda funkcja biznesowa jest niezależną jednostką wdrożeniową z własnym cyklem życia, stosem technologicznym i polityką skalowania. Model ten umożliwia precyzyjne skalowanie tylko tych komponentów, które są przeciążone, co maksymalizuje efektywność wykorzystania zasobów.

1.4.2 Wzorce skalowania

Skalowanie systemów informatycznych można realizować w trzech podstawowych wymiarach:

- **Skalowanie pionowe** (*vertical scaling*) — zwiększanie zasobów pojedynczej instancji (więcej CPU, RAM). Ograniczone fizycznymi możliwościami sprzętu; nie zwiększa odporności na awarie.
- **Skalowanie poziome** (*horizontal scaling*) — zwiększanie liczby instancji obsługujących ruch. Umożliwia rozłożenie obciążenia, zwiększa odporność na awarie, ale wymaga mechanizmów równoważenia obciążenia i koordynacji stanu.
- **Skalowanie automatyczne** (*autoscaling*) — dynamiczne dostosowywanie liczby instancji na podstawie obserwowanych metryk, bez interwencji operatora. Jest to kluczowy mechanizm w systemach chmurowych, gdzie koszty są proporcjonalne do wykorzystania zasobów.

1.5 Platformy orkiestracji kontenerów

1.5.1 Kubernetes — architektura i kluczowe komponenty

Kubernetes (K8s) jest otwartą platformą do automatyzacji wdrażania, skalowania i zarządzania aplikacjami kontenerowymi, pierwotnie opracowaną przez Google i obecnie utrzymaną przez *Cloud Native Computing Foundation* (CNCF).

Architektura *Kubernetes* opiera się na klastrze składającym się z:

- **Control Plane** — warstwa sterowania obejmująca *API server*, *etcd* (magazyn stanu), *scheduler* (przydzielanie Podów do węzłów) oraz *controller manager* (w tym *HorizontalPodAutoscaler*).
- **Worker Nodes** — węzły wykonawcze, na których uruchamiane są Pody za pośrednictwem *kubelet* i *container runtime* (np. *containerd*).

Podstawową jednostką obliczeniową w *Kubernetes* jest **Pod** — grupa jednego lub więcej kontenerów współdzielących przestrzeń sieciową i woluminy. Pody są efemeryczne — mogą być tworzone, niszczone i przenoszone między węzłami.

1.5.2 k3s — lekka dystrybucja Kubernetes

k3s jest certyfikowaną dystrybucją *Kubernetes* zoptymalizowaną pod kątem środowisk o ograniczonych zasobach (IoT, *edge computing*, laboratoria). W porównaniu do standardowego *Kubernetes*, *k3s*:

- Pakuje wszystkie komponenty *Control Plane* w pojedynczy binarny plik wykonywalny.
- Zastępuje *etcd* lekką bazą SQLite (domyślnie) lub osadzonym *etcd*.
- Usuwa nieużywane w mniejszych środowiskach kontrolery i *admission plugins*.
- Używa *containerd* jako domyślnego *container runtime*.

W niniejszej pracy *k3s* został wybrany jako platforma dla lokalnego środowiska badawczego ze względu na niskie wymagania sprzętowe i prostotę instalacji.

1.6 Mechanizmy autoskalowania w Kubernetes

1.6.1 HorizontalPodAutoscaler (HPA)

HorizontalPodAutoscaler jest kontrolerem *Kubernetes*, który automatycznie dostosowuje liczbę replik w *Deployment*, *ReplicaSet* lub *StatefulSet* na podstawie obserwowanych metryk. Algorytm HPA działa według formuły:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil \quad (1.1)$$

HPA w wersji API `autoscaling/v2` obsługuje trzy typy metryk:

1. **Resource metrics** — metryki zasobów kontenera (CPU, pamięć) dostarczane przez *metrics-server*.

2. **Pods metrics** — metryki niestandardowe skojarzone z konkretnymi Podami, dostarczane przez zewnętrzne adaptery (np. *prometheus-adapter*).
3. **Object metrics** — metryki opisujące obiekty *Kubernetes* (np. *Ingress*), również dostarczane przez adaptery.

1.6.2 VerticalPodAutoscaler (VPA)

VerticalPodAutoscaler (VPA) automatycznie dostosowuje *resource requests* i *limits* dla kontenerów w Podzie. W przeciwieństwie do HPA, który zmienia liczbę replik, VPA zmienia ilość zasobów przydzielonych każdej replice. VPA nie jest przedmiotem badań w niniejszej pracy, ale stanowi uzupełniający mechanizm skalowania.

1.6.3 KEDA

KEDA (*Kubernetes Event-Driven Autoscaling*) jest komponentem umożliwiającym skalowanie na podstawie zdarzeń z zewnętrznych systemów (np. długość kolejki *Apache Kafka*, liczba wiadomości w *RabbitMQ*). *KEDA* rozszerza możliwości HPA o skalowanie do zera replik oraz integrację z bogatym ekosystemem *scalerów*. *KEDA* nie jest przedmiotem badań w niniejszej pracy, ale stanowi istotny kierunek dalszych prac.

1.7 Modele SaaS i multi-tenancy

1.7.1 Software as a Service (SaaS)

Model *SaaS* jest jedną z trzech podstawowych kategorii *cloud computing* (obok *IaaS* i *PaaS*), w której dostawca udostępnia użytkownikom gotową aplikację przez Internet, zarządzając całą infrastrukturą, środowiskiem uruchomieniowym i danymi. Kluczowe cechy modelu *SaaS* to:

- **Współdzielenie infrastruktury** — wielu klientów (*tenantów*) korzysta z tej samej instancji aplikacji i infrastruktury.
- **Izolacja danych** — dane każdego tenanta muszą być logicznie odseparowane.
- **Skalowalność** — system musi obsługiwać zmienne obciążenie od wielu tenantów jednocześnie.
- **Rozliczanie** — model subskrypcyjny (miesięczny/roczny) lub *pay-as-you-go*.

1.7.2 Multi-tenancy

Multi-tenancy (wielodostępność) jest architektonicznym wzorcem umożliwiającym obsługę wielu klientów przez pojedynczą instancję systemu. W kontekście *Kubernetes* można wyróżnić kilka poziomów izolacji:

- **Namespace per tenant** — każdy tenant otrzymuje własny *namespace* z izolowanymi zasobami (*ResourceQuota*, *NetworkPolicy*).
- **Application-level multi-tenancy** — tenanci są rozróżniani na poziomie aplikacji (np. przez nagłówek *X-Tenant-ID*), współdzieląc ten sam *namespace*.

- **Cluster per tenant** — każdy tenant otrzymuje dedykowany klaster, co zapewnia najwyższy poziom izolacji przy najwyższym koszcie.

1.8 Przegląd literatury i istniejących badań

Problematyka autoskalowania w *Kubernetes* była przedmiotem licznych badań w ostatnich latach. Poniżej przedstawiono najważniejsze kierunki badawcze i luki, które motywują niniejszą pracę.

1.8.1 Badania nad HPA w Kubernetes

Casalicchio i Perciballi [1] przeprowadzili analizę porównawczą domyślnego HPA *Kubernetes* z *Kubernetes Event-Driven Autoscaling* (KEDA), badając czas reakcji i wykorzystanie zasobów dla obciążeń o różnych profilach. Autorzy wykazali, że KEDA oferuje szybszy czas skalowania przy obciążeniach zdarzeniowych, ale domyślny HPA jest wystarczający dla obciążeń o stałym charakterze.

Nguyen i Kim [2] zaproponowali predykcyjny algorytm skalowania oparty na sieciach LSTM (*Long Short-Term Memory*), który przewiduje przyszłe obciążenie i skaluje proaktywnie, redukując liczbę naruszeń SLA o 30% w porównaniu do reaktywnego HPA. Podejście to wymaga jednak znacznych zasobów obliczeniowych na trenowanie modelu.

1.8.2 Luki badawcze

Na podstawie przeglądu literatury zidentyfikowano następujące luki badawcze:

1. **Brak systematycznego porównania strategii HPA dla różnych profili obciążeniowych** — większość badań testuje HPA na jednorodnym obciążeniu (np. wyłącznie CPU-bound), nie analizując interakcji między profilem obciążeniowym a wyborem metryki.
2. **Ograniczona analiza HPA Memory** — strategia oparta na zużyciu pamięci jest rzadko badana, ponieważ pamięć w większości aplikacji nie koreluje z obciążeniem — niniejsza praca dostarcza empirycznych dowodów na tę tezę.
3. **Brak badań łączących HPA z modelem SaaS** — istniejące badania nad *multi-tenancy* koncentrują się na izolacji danych i bezpieczeństwie, pomijając wpływ wielodostępności na efektywność autoskalowania.
4. **Brak otwartego, odtwarzalnego środowiska badawczego** — większość eksperymentów jest trudna do odtworzenia ze względu na zamknięty kod lub specyficzne dla dostawcy chmury konfiguracje. Niniejsza praca udostępnia kompletne repozytorium kodu, konfiguracji i skryptów.

1.9 Struktura pracy

Praca składa się z siedmiu rozdziałów:

- **Rozdział 1** (niniejszy) — wprowadzenie teoretyczne, motywacja, hipoteza badawcza, przegląd literatury.

- **Rozdział 2** — szczegółowy przegląd technologii wykorzystanych w projekcie: *Kubernetes*, *k3s*, *Prometheus*, *Grafana*, *k6*, *Docker*, *Traefik*, *GKE*.
- **Rozdział 3** — projekt architektury systemu badawczego: macierz badawcza, specyfikacja aplikacji, strategie HPA, infrastruktura monitorująca, plan eksperymentów.
- **Rozdział 4** — implementacja: struktura kodu, implementacja serwisów, konfiguracja *Kubernetes*, monitoring, skrypty automatyzujące, scenariusze testowe.
- **Rozdział 5** — badania eksperymentalne i analiza wyników: metodyka, scenariusze testowe, wyniki, interpretacja, porównanie strategii.
- **Rozdział 6** — rozszerzenie *SaaS Multi-Tenancy*: koncepcja, implementacja izolacji tenantów, testy wielodostępne, wyniki i analiza.
- **Rozdział 7** — podsumowanie, wnioski, rekomendacje, ograniczenia badań i dalsze kierunki prac.

Rozdział 2

Przegląd technologii

Rozdział niniejszy przedstawia technologie wykorzystane w projekcie badawczym, ich architekturę, kluczowe cechy oraz uzasadnienie wyboru każdego komponentu.

2.1 Kubernetes jako platforma orkiestracji

2.1.1 Architektura

Kubernetes (K8s) jest otwartoźródłową platformą do orkiestracji kontenerów, zarządzającą cyklem życia aplikacji w klastrze maszyn. Architektura *Kubernetes* dzieli się na dwie główne warstwy:

Control Plane — warstwa sterowania odpowiedzialna za utrzymanie stanu klastra:

- **kube-apiserver** — centralny punkt komunikacji, wystawia API *Kubernetes* (REST).
- **etcd** — rozproszony magazyn klucz-wartość przechowujący cały stan klastra.
- **kube-scheduler** — przypisuje nowe Pody do węzłów na podstawie dostępnych zasobów i reguł.
- **kube-controller-manager** — uruchamia kontrolery, w tym *HorizontalPodAuto-scaler*, *ReplicaSet controller* i inne.

Worker Nodes — węzły wykonawcze:

- **kubelet** — agent zarządzający Podami na węźle, komunikuje się z *API server*.
- **kube-proxy** — zarządza regułami sieciowymi, implementuje *Service* abstrakcję.
- **container runtime** — uruchamia kontenery (np. *containerd*, *CRI-O*).

2.1.2 Kluczowe obiekty API

- **Pod** — najmniejsza jednostka obliczeniowa; grupa kontenerów ze współdzieloną przestrzenią sieciową i woluminami.
- **Deployment** — deklaratywne zarządzanie replikami Podów z *rolling updates* i *rollback*.
- **Service** — stabilny punkt dostępowy (ClusterIP, NodePort, LoadBalancer) do zestawu Podów.

- **ConfigMap** i **Secret** — zewnętrzna konfiguracja aplikacji.
- **HorizontalPodAutoscaler** — automatyczne skalowanie liczby replik na podstawie metryk.
- **Namespace** — logiczna izolacja zasobów w klastrze.

2.1.3 HorizontalPodAutoscaler (HPA) — szczegółowo

HPA okresowo (domyślnie co 15 sekund) pobiera wartości metryk dla Podów w *Deployment* i oblicza pożądaną liczbę replik. Algorytm skalowania:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil \quad (2.1)$$

Dodatkowo, HPA implementuje mechanizmy stabilizacyjne:

- **Scale-down stabilization window** — domyślnie 5 minut (300 s); HPA czeka tyle czasu przed podjęciem decyzji o skalowaniu w dół, uwzględniając maksymalną wartość replik z tego okna.
- **Tolerance** — domyślnie 10%; metryka musi odchylić się o więcej niż 10% od progu, aby wywołać skalowanie.
- **Cooldown/scaling policies** — od wersji *autoscaling/v2* można definiować polityki ograniczające tempo skalowania (np. maksymalnie 2 repliki na 60 sekund).

2.2 k3s — lekka dystrybucja Kubernetes

2.2.1 Charakterystyka

k3s (wymawiane „k3s”, nie „k-three-s”) jest certyfikowaną przez CNCF dystrybucją *Kubernetes* opracowaną przez *Rancher Labs* (obecnie *SUSE*). Została zaprojektowana dla środowisk o ograniczonych zasobach — *edge computing*, IoT, laboratoria deweloperskie.

2.2.2 Różnice względem standardowego Kubernetes

Aspekt	k3s	Kubernetes (standardowy)
Rozmiar binarki	< 100 MB	> 1 GB (suma komponentów)
Baza stanu	SQLite (domyślnie) / osadzony etcd	etcd (klastr)
Container runtime	containerd (wbudowany)	containerd / CRI-O (osobno)
Instalacja	Jeden binarny plik + systemd unit	kubeadm / kubespray (wiele kroków)
Zależności	Brak (wszystko w binarnym)	etcd, container runtime, CNI

Tabela 2.1 Porównanie *k3s* i standardowego *Kubernetes*

2.2.3 Uzasadnienie wyboru

W niniejszej pracy *k3s* został wybrany z następujących powodów:

- **Ograniczone zasoby sprzętowe** — eksperymenty prowadzone na QNAP NAS z 8 GB RAM; *k3s* zużywa znacząco mniej pamięci niż standardowy *Kubernetes*.
- **Prostota instalacji** — instalacja sprowadza się do jednego polecenia: `curl -sfL https://get.k3s.io | sh -`.
- **Pełna zgodność API** — *k3s* jest certyfikowany przez CNCF, co oznacza, że wszystkie manifesty *Kubernetes* (Deployment, HPA, Service) działają identycznie jak na standardowym klastrze.
- **Niskie zużycie dysku** — *k3s* usuwa nieużywane w mniejszych środowiskach *admission plugins* i kontrolery, redukując rozmiar instalacji.

2.3 Prometheus i Grafana — stos monitorujący

2.3.1 Prometheus

Prometheus jest otwartoźródłowym systemem monitorowania i alertowania, pierwotnie opracowanym w *SoundCloud*, obecnie inkubowanym przez CNCF. Architektura *Prometheusa* opiera się na:

- **Modelu pull** — *Prometheus* aktywnie pobiera (*scrape*) metryki z aplikacji poprzez HTTP w interwałach (w niniejszym projekcie: 15 s).
- **TSDB** (*Time Series Database*) — natywna baza szeregów czasowych przechowująca dane z etykietami (*labels*).
- **PromQL** — funkcyjny język zapytań umożliwiający agregacje, operacje na zakresach czasowych, predykcję.
- **Service Discovery** — automatyczne wykrywanie celów (*targets*) do scrapowania przez integrację z API *Kubernetes*.

W projekcie *Prometheus* pełni podwójną rolę:

1. Zbiera metryki infrastrukturalne z *Kubernetes* (CPU, RAM, liczba replik) poprzez `kubernetes-sd`.
2. Zbiera niestandardowe metryki aplikacyjne z `cpu-service` i `io-service` (RPS, czas przetwarzania, liczba aktywnych zapytań).
3. Udostępnia te metryki przez API dla *prometheus-adapter*, który przekształca je na format zrozumiały dla HPA *Kubernetes*.

2.3.2 Grafana

Grafana jest platformą do wizualizacji danych i tworzenia *dashboardów*, integrującą się z wieloma źródłami danych (m.in. *Prometheus*, *InfluxDB*, *Elasticsearch*). W projekcie wykorzystano:

- **Dashboard „Magisterka — System Overview”** — 16 paneli pokazujących kluczowe metryki: RPS, zużycie CPU per Pod, latency p50/p95/p99, liczbę Podów, stan HPA.

- **Anonimowy dostęp** — uproszczony dostęp do dashboardu w środowisku testowym bez konieczności uwierzytelniania.
- **Dostęp przez port-forward** — Grafana udostępniana lokalnie przez `kubectl port-forward`, eliminując potrzebę zewnętrznego *LoadBalancer*a.

2.3.3 Prometheus Adapter

Prometheus Adapter (`registry.k8s.io/prometheus-adapter/prometheus-adapter:v0.12.0`) jest komponentem implementującym *Kubernetes Custom Metrics API* i *External Metrics API*. Jego rola polega na:

1. Rejestracji jako *APIService* w *Kubernetes*, rozszerzając API o ścieżki `/apis/custom.metrics.k8s`
2. Pobieraniu metryk z *Prometheusa* poprzez zapytania *PromQL*.
3. Serwowaniu tych metryk w formacie oczekiwanym przez HPA.

Konfiguracja adaptera jest przechowywana w `ConfigMap` (`prometheus-adapter-config`) i zawiera reguły mapowania metryk *Prometheusa* na metryki *Kubernetes*. Przykładowa reguła dla CPU-service:

```

1 - seriesQuery: 'cpu_service_http_requests_total{namespace!="",pod!=""}'
2   resources:
3     overrides:
4       namespace: {resource: "namespace"}
5       pod: {resource: "pod"}
6   name:
7     matches: "cpu_service_http_requests_total"
8     as: "http_requests_per_second"
9   metricsQuery: 'sum(rate(cpu_service_http_requests_total{
10     <<.LabelMatchers>>}[1m])) by (<<.GroupBy>>)'

```

Listing 2.1 Reguła mapowania *Prometheus Adapter* dla RPS CPU-service

2.4 k6 — narzędzie do testów obciążeniowych

2.4.1 Charakterystyka

k6 (*Grafana k6*) jest otwartoźródłowym narzędziem do testów obciążeniowych, napisanym w *Go* z interfejsem skryptowym w *JavaScript*. Kluczowe cechy *k6*:

- **Skryptowalność** — testy definiowane w *JavaScript* (ES6), z dostępem do modułów `http`, `check`, `sleep`, `encoding`.
- **Executory** — różne modele obciążenia: `ramping-vus` (stopniowa zmiana liczby VU), `constant-vus` (stała liczba VU), `per-vu-iterations`.
- **Thresholds** — progi akceptacji (np. `p(95) < 10000`) których przekroczenie oznacza niepowodzenie testu.
- **Wydaźność** — pojedyncza instancja *k6* generuje setki-tysiące VU (*Virtual Users*) bez potrzeby rozproszonej architektury.
- **Wyniki** — eksport do JSON, CSV, integracja z *Prometheus* (przez `--out statsd`).

2.4.2 Scenariusze k6 w projekcie

W projekcie zdefiniowano trzy skrypty k6:

- `cpu-load-test.js` — dwa scenariusze: Fibonacci (ramp-up 1→50 VU, steady 120 s) oraz przetwarzanie obrazu (stałe 10 VU).
- `io-load-test.js` — trzy scenariusze: zapytania ze stałym opóźnieniem (ramp-up 1→100 VU), test obciążeniowy z rosnącym opóźnieniem (50→2000 ms), wywołania zewnętrzne (`external_call=true`).
- `combined-scenario.js` — trzy równoległe scenariusze: Fibonacci (30 VU), przetwarzanie obrazu (5 VU), zapytania IO (80 VU).

2.5 Docker i konteneryzacja

2.5.1 Docker

Docker jest platformą do tworzenia, dystrybucji i uruchamiania kontenerów. W projekcie każdy mikroservis jest konteneryzowany za pomocą *Dockerfile*:

- Obraz bazowy: `python:3.12-slim` (minimalny rozmiar, 150 MB).
- Instalacja zależności przez `pip install -r requirements.txt`.
- Uruchomienie przez `uvicorn` (serwer ASGI) z parametrami: `--host 0.0.0.0 --port 8000`.

Wszystkie trzy obrazy Docker (`cpu-service`, `io-service`, `echo-server`) są budowane lokalnie i importowane do *containerd* k3s przez skrypt `deploy-k3s.sh` z użyciem `k3s ctr images import`.

2.5.2 Docker Compose

Docker Compose jest wykorzystywany wyłącznie do lokalnego *developmentu* — szybkie uruchomienie obu serwisów (`cpu-service` i `io-service`) bez pełnego klastra *Kubernetes*. Plik `docker-compose.yaml` w katalogu głównym projektu definiuje serwisy z *Traefik* jako *reverse proxy*.

2.6 Traefik jako Ingress Controller

Traefik jest nowoczesnym *reverse proxy* i *load balancerem* zaprojektowanym dla architektur mikrosługowych. W projekcie *Traefik* pełni rolę:

- **Ingress Controller** w lokalnym środowisku *Docker Compose* — kieruje ruch do odpowiednich serwisów na podstawie nazwy hosta (`cpu.magisterka.osabosa.pl`, `io.magisterka.osabosa.pl`).
- **Automatyczna konfiguracja** — *Traefik* wykrywa kontenery za pomocą *Docker provider* i automatycznie tworzy routing.

W środowisku *Kubernetes* (k3s) *Traefik* nie jest używany — komunikacja między k6 a serwisami odbywa się przez `kubectl port-forward`, co eliminuje potrzebę *Ingress Controllera* w klastrze testowym.

2.7 GKE (Google Kubernetes Engine) — wdrożenie chmurowe

2.7.1 Charakterystyka GKE

Google Kubernetes Engine (GKE) jest zarządzaną usługą *Kubernetes* oferowaną przez *Google Cloud Platform* (GCP). GKE automatyzuje zarządzanie *Control Plane*, aktualizacje, skalowanie węzłów i integrację z ekosystemem GCP.

2.7.2 Porównanie k3s vs GKE

Aspekt	k3s (lokalnie)	GKE (chmura)
Zarządzanie Control Plane	Ręczne (użytkownik)	Automatyczne (GCP)
Koszt	0 (własny sprzęt)	~ 0.25 USD/h (e2-standard-4 + management fee)
Skalowalność węzłów	1 węzeł (QNAP)	Wiele węzłów (node pools)
Opóźnienie sieciowe	< 1 ms (localhost)	~ 1 – 5 ms (intra-zone)
Łatwość utworzenia	<code>curl ... sh</code>	<code>gcloud container clusters create</code>

Tabela 2.2 Porównanie środowiska lokalnego (*k3s*) i chmurowego (*GKE*)

GKE został wybrany jako środowisko chmurowe dla dodatkowej części badawczej (Faza VII) z następujących powodów:

- 300 USD kredytów startowych dla nowych kont GCP (90 dni).
- Najprostsza konfiguracja spośród głównych dostawców chmury (`gcloud container clusters create`).
- Pełna zgodność ze standardowym API *Kubernetes* — manifesty nie wymagają modyfikacji.

2.8 Podsumowanie stosu technologicznego

Warstwa	Technologia	Rola
Język / Framework	Python 3.12 + FastAPI	Implementacja mikroservisów
Serwer ASGI	Uvicorn	Serwer HTTP dla FastAPI
Konteneryzacja	Docker + Docker Compose	Budowa obrazów, lokalny development
Orkiestracja (lokalnie)	k3s v1.28+	Klaster <i>Kubernetes</i> na QNAP
Orkiestracja (chmura)	GKE	Zarządzany <i>Kubernetes</i> w GCP
Monitoring	Prometheus + Grafana	Zbieranie i wizualizacja metryk
Custom Metrics	prometheus-adapter v0.12.0	Mostek Prometheus → HPA
Testy obciążeniowe	k6 v0.48+	Generowanie ruchu, pomiar metryk
Reverse proxy (dev)	Traefik	Routing w Docker Compose

Tabela 2.3 Podsumowanie stosu technologicznego projektu

Rozdział 3

Projekt architektury systemu badawczego

Rozdział niniejszy opisuje projekt architektury systemu badawczego — jego komponenty, strategię skalowania, infrastrukturę monitorującą oraz plan eksperymentów. Wszystkie decyzje projektowe zostały podporządkowane celowi badawczemu: porównaniu skuteczności trzech strategii *HorizontalPodAutoscaler* (HPA) dla dwóch diametralnie różnych profili obciążeniowych.

3.1 Macierz badawcza

Podstawą metodologii badawczej jest macierz 2×3 obejmująca dwie aplikacje o odmiennych profilach obciążeniowych i trzy strategie HPA:

Profil aplikacji	HPA CPU (50%)	HPA Memory (70%)	HPA Custom RPS (100/Pod)
<i>CPU-bound</i> (cpu-service)	cpu-cpu	cpu-memory	cpu-custom
<i>I/O-bound</i> (io-service)	io-cpu	io-memory	io-custom

Tabela 3.1 Macierz badawcza 2×3 — identyfikatory scenariuszy testowych

Każda z sześciu kombinacji jest testowana w odrębnym scenariuszu, co daje łącznie sześć scenariuszy właściwych (plus dwa scenariusze bazowe bez HPA). Macierz umożliwia izolowanie wpływu pojedynczej zmiennej niezależnej (strategia HPA) przy kontrolowanym profilu obciążeniowym.

Oprócz sześciu scenariuszy macierzy badawczej zdefiniowano dwa scenariusze bazowe (*baseline*):

- **baseline-cpu** — *CPU-bound*, brak HPA, 1 replika na stałe.
- **baseline-io** — *I/O-bound*, brak HPA, 1 replika na stałe.

Scenariusze bazowe służą jako punkt odniesienia do oceny, czy HPA w ogóle poprawia wydajność w stosunku do statycznej konfiguracji. Dodatkowo, dla testów integracyjnych zdefiniowano scenariusz łączony (**combined**), w którym oba serwisy są obciążane jednocześnie.

3.2 Aplikacje badawcze

System składa się z trzech mikroserwisów — dwóch podlegających badaniu (`cpu-service`, `io-service`) oraz jednego pomocniczego (`echo-server`). Każdy mikroserwis jest niezależną aplikacją *FastAPI* (Python 3.12) konteneryzowaną za pomocą *Docker* i wdrażaną jako *Deployment* w *Kubernetes*.

3.2.1 CPU-service — obciążenie CPU-bound

`cpu-service` implementuje dwa endpointy obciążające procesor:

- `POST /process` — przetwarzanie obrazu: akceptuje plik graficzny (*multipart form*), wykonuje sekwencję operacji CPU-intensywnych z użyciem biblioteki *Pillow* (m.in. konwersja do skali szarości, filtr `BLUR`, filtr `SHARPEN`, filtr `FIND_EDGES`, ekstrakcja kanałów RGB, `resize`), oblicza entropię Shannona dla każdego kanału, zwraca statystyki przetwarzania (wymiary, entropia, czas).
- `GET /fibonacci` — obliczanie n -tej liczby Fibonacciego rekurencyjnym algorytmem o złożoności $O(2^n)$. Parametr n jest konfigurowalny (domyślnie $n = 30$).

Obie operacje są czysto CPU-intensywne — nie wykonują operacji sieciowych ani dyskowych poza odbiorem żądania i wysłaniem odpowiedzi. Dzięki temu zużycie CPU silnie koreluje z obciążeniem żądaniami.

3.2.2 IO-service — obciążenie I/O-bound

`io-service` implementuje dwa endpointy symulujące operacje wejścia-wyjścia:

- `GET /query` — łańcuch asynchronicznych opóźnień: wykonuje sekwencję wywołań `asyncio.sleep()` z konfigurowalnym opóźnieniem bazowym (`delay`, domyślnie 100 ms), liczbą kroków (`steps`, domyślnie 3) oraz losowym rozrzutem (*jitter*, domyślnie $\pm 20\%$). Całkowity czas odpowiedzi jest sumą opóźnień, a procesor pozostaje bezczynny podczas oczekiwania.
- `GET /upstream` — wywołanie zewnętrzne: przekazuje żądanie do `echo-server` i zwraca odpowiedź z dodatkowym opóźnieniem symulującym przetwarzanie.

Serwis wykorzystuje `asyncio.Semaphore(50)` do ograniczenia liczby współbieżnych zapytań, co zapobiega wyczerpaniu zasobów systemowych przy bardzo wysokim obciążeniu.

3.2.3 Echo-server — serwis pomocniczy

`echo-server` jest minimalistycznym serwisem pełniącym rolę zależności zewnętrznej dla `io-service`. Udostępnia dwa endpointy:

- `GET /health` — *health check*.
- `POST /echo` — zwraca kopię otrzymanego żądania (nagłówki, body, parametry), umożliwiając weryfikację poprawnego przekazywania danych.

Echo-server jest wyłączony z macierzy badawczej — służy wyłącznie jako cel wywołań w scenariuszu `external_calls` dla `io-service`.

3.2.4 Memory-service — serwis pamięciowy (dodatkowy)

W ramach rozwoju projektu zaimplementowano również `memory-service` — serwis z pamięciową bazą klucz-wartość (*in-memory cache*) symulujący obciążenie typu *memory-bound*. Udostępnia endpointy `GET/POST/DELETE /cache`, `POST /cache/fill` (wypełnienie cache danymi testowymi o kontrolowanym rozmiarze) oraz `GET /stats` (statystyki trafień). `memory-service` nie został włączony do głównej macierzy badawczej na obecnym etapie prac — stanowi rozszerzenie do potencjalnych przyszłych badań.

3.3 Strategie HPA

Dla każdej aplikacji badawczej zdefiniowano trzy warianty *HorizontalPodAutoscaler*, różniące się wyłącznie typem metryki. Wszystkie HPA współdzielą następujące parametry:

- `minReplicas`: 2
- `maxReplicas`: 10
- `apiVersion`: `autoscaling/v2`

3.3.1 HPA CPU (Resource Metric)

Strategia oparta na zużyciu CPU — HPA monitoruje średnie zużycie procesora jako procent *resource request* dla wszystkich Podów w *Deployment*. Wartość progowa: 50%. Oznacza to, że jeśli średnie zużycie CPU przekracza 50% `requests` (czyli 125 m dla `cpu-service`, 50 m dla `io-service`), HPA dodaje repliki.

```

1 metrics:
2   - type: Resource
3     resource:
4       name: cpu
5       target:
6         type: Utilization
7         averageUtilization: 50

```

Listing 3.1 Konfiguracja HPA CPU dla `cpu-service` (plik `k8s/cpu-service/hpa-cpu.yaml`)

Strategia CPU jest najczęściej stosowaną i domyślnie rekomendowaną w ekosystemie *Kubernetes*. Jej skuteczność zależy od tego, czy zużycie CPU koreluje z obciążeniem aplikacji.

3.3.2 HPA Memory (Resource Metric)

Strategia oparta na zużyciu pamięci — HPA monitoruje średnie zużycie pamięci jako procent *resource request*. Wartość progowa: 70%. Strategia ta jest rzadziej stosowana, ponieważ większość aplikacji alokuje pamięć w sposób nieelastyczny (po uruchomieniu lub przy pierwszym żądaniu), co sprawia, że metryka pamięci nie odzwierciedla bieżącego obciążenia.

```

1 metrics:
2   - type: Resource
3     resource:
4       name: memory

```

```

5     target:
6         type: Utilization
7         averageUtilization: 70

```

Listing 3.2 Konfiguracja HPA Memory dla io-service (plik k8s/io-service/hpa-memory.yaml)

Hipoteza badawcza przewiduje, że HPA Memory będzie nieskuteczne dla obu profili obciążeniowych — ani CPU-bound, ani I/O-bound nie powinny generować istotnych wahań zużycia pamięci skorelowanych z ruchem.

3.3.3 HPA Custom RPS (Pods Metric)

Strategia oparta na niestandardowej metryce `http_requests_per_second` — liczbie żądań HTTP na sekundę na Pod. Metryka jest dostarczana przez *prometheus-adapter*, który transformuje liczniki *Prometheusa* (`cpu_service_http_requests_total`, `io_service_http_requests`) na chwilową wartość RPS za pomocą funkcji `rate()` [1m]. Wartość progowa: 100 żądań na sekundę na Pod.

```

1 metrics:
2   - type: Pods
3     pods:
4       metric:
5         name: http_requests_per_second
6       target:
7         type: AverageValue
8         averageValue: "100"

```

Listing 3.3 Konfiguracja HPA Custom RPS (plik k8s/cpu-service/hpa-custom.yaml)

Strategia RPS jest bezpośrednio powiązana z obciążeniem żadaniami — niezależnie od tego, czy żądania są CPU-intensywne, czy I/O-intensywne, liczba żądań na sekundę rośnie proporcjonalnie do ruchu. Dzięki temu HPA Custom RPS powinna być skuteczna dla obu profili obciążeniowych.

3.4 Infrastruktura monitorująca

3.4.1 Prometheus

Prometheus pełni rolę centralnego systemu zbierania metryk. Jest skonfigurowany do scrapowania:

- **Metryk infrastrukturalnych** — z `kubernetes-sd` (CPU, RAM, liczba replik, stan Podów) co 15 sekund.
- **Metryk aplikacyjnych** — z endpointów `/metrics` wystawianych przez `cpu-service`, `io-service` i `echo-server` (RPS, czas przetwarzania, liczba aktywnych zapytań, metryki niestandardowe).

3.4.2 Grafana

Grafana udostępnia dashboard „Magisterka — System Overview” zawierający 16 paneli wizualizujących kluczowe metryki systemu w czasie rzeczywistym (odświeżanie co 10 sekund):

- RPS dla `cpu-service` i `io-service` (panele typu `stat` z miniwykresem).
- Zużycie CPU na Pod (panele typu `timeseries`).
- Zużycie pamięci na Pod.
- Liczba replik per *Deployment*.
- Opóźnienia (*latency*): p50, p95, p99.
- Liczba żądań w toku (*in-flight*).
- Stan HPA (pożądana vs. aktualna liczba replik).

3.4.3 Prometheus Adapter

Prometheus Adapter (wersja v0.12.0) implementuje *Custom Metrics API* w *Kubernetes*, umożliwiając HPA odczytywanie metryk z *Prometheusa*. Konfiguracja adaptera (przechowywana w `ConfigMap` `prometheus-adapter-config`) definiuje cztery reguły mapowania:

1. `cpu_service_http_requests_total` → `http_requests_per_second` (przez `rate()` [1m]).
2. `io_service_http_requests_total` → `http_requests_per_second` (przez `rate()` [1m]).
3. `cpu_service_http_requests_inprogress` → `http_requests_inflight` (wartość chwilowa).
4. `io_service_http_requests_inprogress` → `http_requests_inflight` (wartość chwilowa).

Reguły mapowania wykorzystują mechanizm `<<.LabelMatchers>>` i `<<.GroupBy>>` do dynamicznego filtrowania po namespace i Podzie, co jest wymagane przez *Custom Metrics API*.

3.5 Topologia wdrożeniowa

Wszystkie komponenty systemu są wdrażane w pojedynczym namespace `magisterka`. Topologia obejmuje:

- **3 Deploymenty** aplikacyjne: `cpu-service`, `io-service`, `echo-server` (plus opcjonalnie `memory-service`).
- **3 Serwisy** typu `ClusterIP`: każdy kieruje ruch na port 80 do `targetPort 8000` odpowiedniego *Deploymentu*.
- **6 HPA** (po 3 na serwis badawczy): pliki `hpa-cpu.yaml`, `hpa-memory.yaml`, `hpa-custom.yaml`.
- **Monitoring**: `prometheus` + `grafana` + `prometheus-adapter` w tym samym namespace.
- **Ingress** (*Traefik*): routing na podstawie nazwy hosta (`cpu-magisterka.k3s.local` → `cpu-service`, `io-magisterka.k3s.local` → `io-service`).

W środowisku testowym ruch z *k6* do serwisów jest kierowany przez `kubectl port-forward`, co eliminuje potrzebę zewnętrznego *LoadBalancer*a i zapewnia minimalne opóźnienie sieciowe (< 1 ms, lokalny `localhost`).

3.5.1 Zasoby przydzielone serwisom

Serwis	CPU request	CPU limit	Memory request / limit
cpu-service	250 m	1000 m	256 Mi / 512 Mi
io-service	100 m	500 m	128 Mi / 256 Mi
echo-server	50 m	200 m	64 Mi / 128 Mi
memory-service	200 m	500 m	256 Mi / 1024 Mi

Tabela 3.2 Zasoby przydzielone serwisom (*resource requests* i *limits*)

Różnica w przydziale zasobów między `cpu-service` a `io-service` odzwierciedla odmienne profile obciążeniowe: `cpu-service` potrzebuje więcej CPU (250 m vs. 100 m) i pamięci (256 Mi vs. 128 Mi) ze względu na operacje przetwarzania obrazu.

3.6 Plan eksperymentów

3.6.1 Scenariusze testowe

Eksperyment obejmuje 8 scenariuszy uruchamianych sekwencyjnie:

1. `baseline-cpu` — CPU-bound, brak HPA, 1 replika.
2. `baseline-io` — I/O-bound, brak HPA, 1 replika.
3. `cpu-cpu` — CPU-bound + HPA CPU (50%).
4. `cpu-memory` — CPU-bound + HPA Memory (70%).
5. `cpu-custom` — CPU-bound + HPA Custom RPS (100/Pod).
6. `io-cpu` — I/O-bound + HPA CPU (50%).
7. `io-memory` — I/O-bound + HPA Memory (70%).
8. `io-custom` — I/O-bound + HPA Custom RPS (100/Pod).

3.6.2 Procedura eksperymentalna

Każdy scenariusz jest wykonywany według ściśle określonej procedury, zaimplementowanej w skrypcie `scripts/run-tests.sh`:

1. **Przełączenie HPA** — usunięcie wszystkich istniejących HPA dla docelowego serwisu i nałożenie pliku HPA odpowiadającego testowanej strategii.
2. **Reset Deploymentu** — skalowanie *Deploymentu* do 1 repliki i oczekiwanie na gotowość (`rollout status`).
3. **Uruchomienie testu k6** — wykonanie właściwego skryptu obciążeniowego.
4. **Zapis wyników** — wyniki k6 (JSON + tekstowe podsumowanie) zapisywane do katalogu `results/<scenariusz>/`.

5. **Powtórzenie** — kroki 1–4 powtarzane 5-krotnie dla każdego scenariusza.
6. **Cooldown** — po każdym scenariuszu: skalowanie obu serwisów do 1 repliki, usunięcie wszystkich HPA, oczekiwanie 120 sekund na stabilizację klastra.

3.6.3 Profile obciążeniowe k6

Każdy scenariusz wykorzystuje dedykowany skrypt *k6* dopasowany do profilu obciążeniowego:

CPU-bound (skrypt `cpu-load-test.js`):

- **Scenariusz Fibonacci**: ramp-up od 1 do 50 VU w ciągu 60 sekund, następnie stałe obciążenie 50 VU przez 120 sekund.
- **Scenariusz przetwarzanie obrazu**: stałe 10 VU przez 180 sekund, każde żądanie wysyła losowo wygenerowany obraz PNG (100×100 pikseli) do `POST /process`.

I/O-bound (skrypt `io-load-test.js`):

- **Scenariusz stałych zapytań**: ramp-up od 1 do 100 VU w ciągu 60 sekund, stałe obciążenie przez 120 sekund, `delay=100ms`, `steps=3`.
- **Scenariusz stres-test**: stałe 20 VU, `delay` rosnący od 50 ms do 2000 ms w krokach co 30 sekund.
- **Scenariusz wywołań zewnętrznych**: stałe 20 VU, `external_call=true`, żądania do `GET /upstream`.

Scenariusz łączony (skrypt `combined-scenario.js`):

- Fibonacci: 30 VU (ramp-up 30 s, steady 180 s).
- Przetwarzanie obrazu: 5 VU (stałe, 180 s).
- Zapytania IO: 80 VU (ramp-up 60 s, steady 180 s).

Wszystkie skrypty definiują progi akceptacji (*thresholds*): $p(95) < 5000$ ms dla scenariuszy CPU, $p(95) < 10000$ ms dla scenariuszy IO, oraz `http_req_failed` $< 1\%$ (CPU) / $< 5\%$ (IO).

3.6.4 Metryki badawcze

Dla każdego scenariusza zbierane są następujące metryki:

- **Opóźnienie (*latency*)** — $p(50)$, $p(95)$, $p(99)$ czasu odpowiedzi HTTP (ms).
- **Przepustowość (*throughput*)** — liczba żądań na sekundę (RPS) obsłużonych przez system.
- **Czas do skalowania (*time-to-scale*)** — czas od przekroczenia progu HPA do osiągnięcia docelowej liczby replik.
- **Efektywność skalowania (*scaling efficiency*)** — stosunek zmiany przepustowości do zmiany liczby replik.

- **Stopa błędów (*error rate*)** — odsetek żądań zakończonych kodem HTTP ≥ 400 .
- **Liczba replik w czasie** — przebieg czasowy skalowania (dane z *Prometheusa*).
- **Zużycie CPU i pamięci** — średnie i szczytowe zużycie zasobów na Pod.

3.7 Podsumowanie architektury

Zaprojektowany system badawczy umożliwia kontrolowane, powtarzalne eksperymenty porównujące skuteczność trzech strategii HPA dla dwóch profili obciążeniowych. Kluczowe decyzje architektoniczne:

1. **Izolacja zmiennych** — każdy scenariusz testuje dokładnie jedną kombinację (profil $\times$ strategia), przy wyizolowanym serwisie (port-forward) i ustabilizowanym klastrze (cooldown).
2. **Powtarzalność** — każdy scenariusz jest powtarzany 5-krotnie, co umożliwia analizę statystyczną i eliminację wpływu anomalii.
3. **Automatyzacja** — cały proces (przełączanie HPA, reset Deploymentów, uruchamianie k6, zbieranie wyników) jest w pełni zautomatyzowany przez skrypt `run-tests.sh`.
4. **Odtwarzalność** — wszystkie manifesty *Kubernetes*, skrypty k6 i konfiguracje są przechowywane w repozytorium, co umożliwia odtworzenie środowiska na dowolnym klastrze *Kubernetes*.

Rozdział 4

Implementacja

Rozdział niniejszy opisuje implementację systemu badawczego — strukturę kodu, implementację mikroservisów, konfigurację *Kubernetes*, infrastrukturę monitorującą oraz skrypty automatyzujące. Wszystkie listingi odwołują się do rzeczywistych plików w repozytorium projektu.

4.1 Struktura repozytorium

Kod projektu jest zorganizowany w następującej strukturze katalogów:

```
magisterka/
  apps/
    cpu-service/      # CPU-bound microservice
    io-service/       # I/O-bound microservice
    echo-server/      # Auxiliary echo service
    memory-service/   # Memory-bound microservice (extension)
  k8s/
    base/             # Namespace
    cpu-service/      # Deployment + Service + HPA x3
    io-service/       # Deployment + Service + HPA x3
    echo-service/     # Deployment + Service
    memory-service/   # Deployment + Service + HPA
    monitoring/       # Prometheus, Grafana, Prometheus Adapter
    ingress.yaml      # Traefik Ingress
  monitoring/
    dashboards/       # Grafana dashboard JSON
  scripts/
    deploy-k3s.sh     # Full deployment pipeline
    run-tests.sh      # Test orchestration (8 scenarios)
    run-all.sh        # deploy + tests combined
  tests/
    k6/               # k6 load test scripts
  docs/
    prd.md            # Product Requirements Document
    docker-compose.yaml # Local development environment
```

4.2 Implementacja mikroservisów

Każdy mikroservis jest zaimplementowany jako niezależna aplikacja *FastAPI* (Python 3.12), konteneryzowana za pomocą *Docker* i uruchamiana przez *Uvicorn* (serwer ASGI).

4.2.1 Wspólne cechy implementacyjne

Wszystkie serwisy dzielą następujące cechy:

- **FastAPI + Uvicorn** — framework *FastAPI* 0.115.6 z serwerem *Uvicorn* 0.34.0. Wybór *FastAPI* podyktowany jest natywną obsługą asynchroniczności (`async/await`), automatyczną generacją dokumentacji OpenAPI oraz lekkością.
- **Dockerfile** — identyczny wzorzec dla wszystkich serwisów: obraz bazowy `python:3.12-slim`, instalacja zależności przez `pip install --no-cache-dir`, aplikacja kopiowana jako pojedynczy plik `main.py`.
- **Prometheus metrics** — automatyczna instrumentacja przez `prometheus-fastapi-instrument` 7.0.0, która wystawia metryki HTTP (liczniki żądań, histogramy czasu odpowiedzi, liczba żądań w toku) na endpointzie `/metrics`.
- **Health check** — endpoint GET `/health` zwracający `{"status": "ok", "service": "..."}` , wykorzystywany przez *liveness* i *readiness probes* w *Kubernetes*.
- **CORS** — middleware `CORSMiddleware` z zezwoleniem na wszystkie źródła (`allow_origins=["*"]`) dla wygody testowania.

```

1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY main.py .
6 EXPOSE 8000
7 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Listing 4.1 Dockerfile dla `cpu-service` (wzorzec wspólny dla wszystkich serwisów)

4.2.2 CPU-service

`cpu-service` implementuje dwa endpointy obciążające procesor:

POST `/process` — przetwarzanie obrazu

Endpoint akceptuje obraz w formacie *multipart form-data*, wykonuje 5-etapowy potok przetwarzania CPU-intensywnego z użyciem biblioteki *Pillow* 11.1.0:

1. Konwersja do skali szarości (`image.convert("L")`).
2. Zastosowanie filtru BLUR.
3. Zastosowanie filtru SHARPEN.
4. Zastosowanie filtru FIND_EDGES.

5. Ekstrakcja kanałów RGB i resize do 100×100 pikseli.

Po przetworzeniu, dla każdego kanału obliczana jest entropia Shannona według wzoru:

$$H = - \sum_{i=0}^{255} p_i \log_2(p_i), \quad \text{gdzie } p_i = \frac{\text{histogram}[i]}{\text{total_pixels}} \quad (4.1)$$

```

1 -----
2 def _shannon_entropy(histogram: list[int], total_pixels: int) -> float:
3     """Calculate Shannon entropy from a pixel-value histogram."""
4     entropy = 0.0
5     for count in histogram:
6         if count == 0:
7             continue
8         prob = count / total_pixels
9         entropy -= prob * math.log2(prob)
10    return entropy

```

Listing 4.2 Funkcja `shannon_entropy()` w `cpu-service`

GET /fibonacci — ciąg Fibonacciego

Endpoint oblicza n -tą liczbę Fibonacciego rekurencyjnym algorytmem o złożoności $O(2^n)$. Parametr n jest pobierany z *query string* (domyślnie 30). Wybór rekurencyjnej implementacji jest celowy — algorytm ten jest skrajnie nieefektywny obliczeniowo, co gwarantuje wysokie zużycie CPU przy niewielkiej liczbie żądań.

```

1 -----
2 def _fib_recursive(n: int) -> int:
3     """Compute the n-th Fibonacci number using naive double recursion.
4
5     This is intentionally O(2^n) - the worst possible implementation -
6     to serve as a pure CPU-burn endpoint for scalability experiments.
7     """
8     if n <= 1:
9         return n
10    return _fib_recursive(n - 1) + _fib_recursive(n - 2)

```

Listing 4.3 Rekurencyjna implementacja ciągu Fibonacciego w `cpu-service`

Metryki niestandardowe

`cpu-service` wystawia dwie niestandardowe metryki *Prometheus*:

- `cpu_service_images_processed_total` — *Counter* zliczający przetworzone obrazy.
- `cpu_service_processing_duration_seconds` — *Histogram* czasu przetwarzania obrazu.

4.2.3 IO-service

`io-service` implementuje dwa endpointy symulujące operacje I/O:

GET /query — łańcuch opóźnień asynchronicznych

Endpoint wykonuje sekwencję wywołań `asyncio.sleep()` z konfigurowalnymi parametrami:

- `delay` (domyślnie 100 ms) — bazowe opóźnienie pojedynczego kroku.
- `steps` (domyślnie 3) — liczba kroków w łańcuchu.
- `jitter` (domyślnie 0.2) — losowy rozrzut $\pm 20\%$ wokół `delay`.

Całkowity czas odpowiedzi wynosi $\text{steps} \times \text{delay} \times (1 \pm \text{jitter})$, a procesor pozostaje bezczynny podczas oczekiwania (`asyncio.sleep` zwalnia wątek). Dzięki temu zużycie CPU nie koreluje z liczbą żądań — serwis jest czysto *I/O-bound*.

```

1  -----
2  @app.get("/query", response_model=QueryResponse)
3  async def query(
4      delay: int = Query(200, ge=0, le=5000, description="Base delay per step
5          in milliseconds"),
6      steps: int = Query(3, ge=1, le=10, description="Number of simulated
7          calls in the chain"),
8      external_call: bool = Query(
9          True,
10         description="If true, call /upstream via HTTP; if false, use
11             asyncio.sleep",
12     ),
13 ):
14     """Simulate a chain of I/O-bound operations.
15
16     Each step introduces a delay with  $\pm 20\%$  random jitter. When
17     ``external_call=true`` the step performs an HTTP GET to ``/upstream``;
18     otherwise it uses ``asyncio.sleep``.
19
20     Returns timing breakdown for every step plus the aggregate.
21     """
22     _validate_query_params(delay, steps)
23
24     semaphore: asyncio.Semaphore = app.state.concurrency_semaphore
25     http_client: httpx.AsyncClient = app.state.http_client
26
27     # Track concurrency via the gauge - increment once inside the semaphore
28     await semaphore.acquire()
29     io_service_concurrent_queries.inc()
30
31     try:
32         step_times_ms: list[float] = []
33
34         total_start = time.perf_counter()
35
36         for _step_idx in range(steps):
37             # Apply  $\pm 20\%$  random jitter to the base delay
38             jitter = random.uniform(0.8, 1.2)
39             actual_delay_ms = delay * jitter
40
41             step_start = time.perf_counter()
42
43             if external_call:
44                 # Real HTTP call to the upstream helper endpoint

```



```

42         try:
43             response = await http_client.get(
44                 f"{_UPSTREAM_BASE_URL}/upstream",
45                 params={"delay": int(actual_delay_ms)},
46             )
47             response.raise_for_status()
48         except httpx.HTTPError as exc:
49             # If the upstream call fails, still record the timing
50             # and propagate a meaningful error to the caller
51             step_elapsed = time.perf_counter() - step_start
52             step_times_ms.append(round(step_elapsed * 1000, 2))
53             raise HTTPException(
54                 status_code=502,
55                 detail=f"Upstream call failed at step {_step_idx +
56                     1}: {exc}",
57             )
58             step_elapsed = time.perf_counter() - step_start
59             # Observe upstream duration in the histogram (in seconds)
60             io_service_upstream_duration_seconds.observe(step_elapsed)

```

Listing 4.4 Endpoint GET /query w io-service — łańcuch opóźnień asynchronicznych

GET /upstream — wywołanie zewnętrzne

Endpoint przekazuje żądanie do echo-server za pomocą `httpx.AsyncClient` z timeoutem 10 sekund i zwraca odpowiedź z dodatkowym opóźnieniem symulującym przetwarzanie. Parametry: `delay` (domyślnie 50 ms), `jitter` (domyślnie 0.1).

Kontrola współbieżności

Serwis wykorzystuje `asyncio.Semaphore(50)` do ograniczenia liczby współbieżnych zapytań, co zapobiega wyczerpaniu zasobów systemowych przy bardzo wysokim obciążeniu.

```

1  -----
2  @asynccontextmanager
3  async def lifespan(app: FastAPI):
4      """Startup: create shared httpx client and concurrency semaphore.
5      Shutdown: close the httpx client."""
6      app.state.http_client = httpx.AsyncClient(
7          timeout=httpx.Timeout(30.0),
8          limits=httpx.Limits(max_keepalive_connections=20, max_connections
9              =100),
10         )
11         app.state.concurrency_semaphore = asyncio.Semaphore(50)
12         try:
13             yield
14         finally:
15             await app.state.http_client.aclose()

```

Listing 4.5 Inicjalizacja semafora współbieżności w io-service

Metryki niestandardowe

io-service wystawia trzy niestandardowe metryki:

- `io_service_queries_total` — *Counter* zliczający zapytania.

- `io_service_upstream_duration_seconds` — *Histogram* czasu wywołań zewnętrznych.
- `io_service_concurrent_queries` — *Gauge* liczby aktualnie przetwarzanych zapytań.

4.2.4 Echo-server

`echo-server` jest minimalistycznym serwisem pomocniczym z dwoma endpointami: `GET /health` i `POST /echo`. Endpoint `/echo` zwraca kopię otrzymanego żądania (nagłówki, *body*, parametry), umożliwiając weryfikację poprawnego przekazywania danych przez `io-service` w scenariuszu `external_calls`.

Serwis korzysta z biblioteki `httpx` do opcjonalnych wywołań zwrotnych do `io-service` i rejestruje niestandardowe liczniki `echo_server_echoes_total` oraz `echo_server_upstream_calls_t`.

4.2.5 Memory-service

`memory-service` implementuje pamięciową bazę klucz-wartość (*in-memory cache*) z limitem 10000 wpisów (`MAX_CACHE_ENTRIES`). Udostępnia pełne CRUD dla cache: `GET /cache`, `GET /cache/{key}`, `POST /cache/{key}`, `DELETE /cache/{key}`, `DELETE /cache`.

Endpoint badawczy `POST /cache/fill` wypełnia cache danymi testowymi o kontrolowanym rozmiarze — parametry `sessions` (liczba wpisów, 1–5000) i `size_kb` (rozmiar pojedynczego wpisu w KB, 1–5000). Wartości są generowane przez `random.choices` z alfabetu ASCII, co zapobiega optymalizacji *string-interning* Pythona.

```

1 -----
2 def _generate_value(size_kb: int) -> str:
3     """Generate a string of approximately *size_kb* kilobytes using random
4         ASCII letters.
5
6     Uses ``random.choices`` to avoid Python's string-interning optimisation
7     that would make ``'x' * N`` consume less memory than expected.
8     """
9     length = size_kb * 1024
10    return "".join(random.choices(string.ascii_letters, k=length))

```

Listing 4.6 Generowanie wartości testowych w `memory-service`

Endpoint `GET /stats` zwraca statystyki cache: liczbę wpisów, szacowany rozmiar, liczbę trafień/chybień, współczynnik trafień (*hit ratio*) oraz średnią liczbę dostępów.

4.3 Konfiguracja Kubernetes

4.3.1 Namespace i struktura zasobów

Wszystkie zasoby *Kubernetes* są wdrażane w dedykowanym *namespace* `magisterka`. Plik `k8s/base/namespace.yaml` definiuje ten namespace.

4.3.2 Deploymenty

Każdy *Deployment* definiuje:

- `replicas: 2` — początkowa liczba replik.

- `imagePullPolicy: IfNotPresent` — obrazy są importowane lokalnie do *containerd* k3s.
- `livenessProbe` i `readinessProbe` — sprawdzanie GET `/health` co 10 sekund, z początkowym opóźnieniem 5 sekund.
- `resources` — *requests* i *limits* CPU i pamięci (Tabela 3.2).

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: cpu-service
5   namespace: magisterka
6   labels:
7     app: cpu-service
8     tier: backend
9 spec:
10   replicas: 2
11   selector:
12     matchLabels:
13       app: cpu-service
14   template:
15     metadata:
16       labels:
17         app: cpu-service
18         tier: backend
19     spec:
20       containers:
21       - name: cpu-service
22         image: cpu-service:latest
23         imagePullPolicy: IfNotPresent
24         ports:
25         - containerPort: 8000
26         resources:
27           requests:
28             cpu: 250m
29             memory: 256Mi
30           limits:
31             cpu: 1000m
32             memory: 512Mi
33         livenessProbe:
34           httpGet:
35             path: /health
36             port: 8000
37           initialDelaySeconds: 5
38           periodSeconds: 10
39         readinessProbe:
40           httpGet:
41             path: /health
42             port: 8000
43           initialDelaySeconds: 5
44           periodSeconds: 10
```

Listing 4.7 Deployment `cpu-service` (plik `k8s/cpu-service/deployment.yaml`)

4.3.3 HPA

Dla `cpu-service` i `io-service` zdefiniowano po trzy warianty HPA, różniące się wyłącznie typem metryki. Każdy HPA używa API `autoscaling/v2` i ma zakres 2–10 replik.

HPA CPU

```
1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: cpu-service-hpa
5   namespace: magisterka
6   labels:
7     app: cpu-service
8 spec:
9   scaleTargetRef:
10    apiVersion: apps/v1
11    kind: Deployment
12    name: cpu-service
13   minReplicas: 2
14   maxReplicas: 10
15   metrics:
16   - type: Resource
17     resource:
18       name: cpu
19       target:
20         type: Utilization
21         averageUtilization: 50
```

Listing 4.8 HPA CPU dla `cpu-service` (plik `k8s/cpu-service/hpa-cpu.yaml`)

HPA Memory

```
1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: cpu-service-hpa-memory
5   namespace: magisterka
6   labels:
7     app: cpu-service
8     hpa-strategy: memory
9 spec:
10   scaleTargetRef:
11    apiVersion: apps/v1
12    kind: Deployment
13    name: cpu-service
14   minReplicas: 2
15   maxReplicas: 10
16   metrics:
17   - type: Resource
18     resource:
19       name: memory
20       target:
21         type: Utilization
22         averageUtilization: 70
```

Listing 4.9 HPA Memory dla `cpu-service` (plik `k8s/cpu-service/hpa-memory.yaml`)

HPA Custom RPS

```
1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: cpu-service-hpa-custom
5   namespace: magisterka
6   labels:
7     app: cpu-service
8     hpa-strategy: custom-rps
9 spec:
10  scaleTargetRef:
11    apiVersion: apps/v1
12    kind: Deployment
13    name: cpu-service
14  minReplicas: 2
15  maxReplicas: 10
16  metrics:
17    - type: Pods
18      pods:
19        metric:
20          name: http_requests_per_second
21        target:
22          type: AverageValue
23          averageValue: "100"
```

Listing 4.10 HPA Custom RPS dla cpu-service (plik k8s/cpu-service/hpa-custom.yaml)

4.3.4 Services

Wszystkie serwisy *Kubernetes* są typu ClusterIP, mapując port 80 na targetPort 8000:

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: io-service
5   namespace: magisterka
6   labels:
7     app: io-service
8 spec:
9   type: ClusterIP
10  selector:
11    app: io-service
12  ports:
13    - port: 80
14      targetPort: 8000
15      protocol: TCP
```

Listing 4.11 Service dla io-service (plik k8s/io-service/service.yaml)

4.3.5 Ingress

Traefik Ingress kieruje ruch na podstawie nazwy hosta:

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
```

```
3 metadata:
4   name: magisterka-ingress
5   namespace: magisterka
6   labels:
7     app: magisterka
8   annotations:
9     traefik.ingress.kubernetes.io/router.entrypoints: web
10 spec:
11   ingressClassName: traefik
12   rules:
13     - host: cpu-magisterka.k3s.local
14       http:
15         paths:
16           - path: /
17             pathType: Prefix
18             backend:
19               service:
20                 name: cpu-service
21                 port:
22                   number: 80
23     - host: io-magisterka.k3s.local
24       http:
25         paths:
26           - path: /
27             pathType: Prefix
28             backend:
29               service:
30                 name: io-service
31                 port:
32                   number: 80
```

Listing 4.12 Ingress *Traefik* (plik `k8s/ingress.yaml`)

4.4 Infrastruktura monitorująca

4.4.1 Prometheus

Prometheus jest wdrożony w namespace `magisterka` i skonfigurowany do automatycznego wykrywania celów przez `kubernetes_sd_configs` z filtrowaniem po *namespace* i *annotation* `prometheus.io/scrape: "true"`.

4.4.2 Grafana

Grafana jest wdrożona z anonimowym dostępem i predefiniowanym źródłem danych *Prometheus*. Dashboard „Magisterka — System Overview” (UID: `magisterka-overview`) zawiera 16 paneli, odświeżanych co 10 sekund, pokazujących kluczowe metryki w czasie rzeczywistym.

4.4.3 Prometheus Adapter

Prometheus Adapter v0.12.0 implementuje *Custom Metrics API*. Konfiguracja w `ConfigMap` definiuje reguły *PromQL* dla metryk RPS i *in-flight* dla obu serwisów badawczych:

```
1 apiVersion: v1
2 kind: ConfigMap
```

```

3 metadata:
4   name: prometheus-adapter-config
5   namespace: magisterka
6   labels:
7     app: prometheus-adapter
8     tier: monitoring
9 data:
10  config.yaml: |
11    rules:
12      default: false
13
14      # --- CPU-service custom metrics ---
15      - seriesQuery: 'cpu_service_http_requests_total{namespace!="",pod
16        !=""}'
17        resources:
18          overrides:
19            namespace: {resource: "namespace"}
20            pod: {resource: "pod"}
21        name:
22          matches: "cpu_service_http_requests_total"
23          as: "http_requests_per_second"
24        metricsQuery: 'sum(rate(cpu_service_http_requests_total{<<.
25          LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
26
27      - seriesQuery: 'cpu_service_http_requests_inprogress{namespace!="",
28        pod!=""}'
29        resources:
30          overrides:
31            namespace: {resource: "namespace"}
32            pod: {resource: "pod"}
33        name:
34          matches: "cpu_service_http_requests_inprogress"
35          as: "http_requests_inflight"
36        metricsQuery: 'sum(cpu_service_http_requests_inprogress{<<.
37          LabelMatchers>>}) by (<<.GroupBy>>)'
38
39      # --- IO-service custom metrics ---
40      - seriesQuery: 'io_service_http_requests_total{namespace!="",pod!=""}
41        ,
42        resources:
43          overrides:
44            namespace: {resource: "namespace"}
45            pod: {resource: "pod"}
46        name:
47          matches: "io_service_http_requests_total"
48          as: "http_requests_per_second"
49        metricsQuery: 'sum(rate(io_service_http_requests_total{<<.
50          LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
51
52      - seriesQuery: 'io_service_http_requests_inprogress{namespace!="",pod
53        !=""}'
54        resources:
55          overrides:
56            namespace: {resource: "namespace"}
57            pod: {resource: "pod"}
58        name:
59          matches: "io_service_http_requests_inprogress"
60          as: "http_requests_inflight"
61        metricsQuery: 'sum(io_service_http_requests_inprogress{<<.

```

```

55         LabelMatchers>>}) by (<<.GroupBy>>)'
56
57     # --- Resource metrics (passthrough to metrics-server) ---
58     - seriesQuery: '{__name__=~"^container_.*",namespace!="",pod!=""}'
59     resources:
60         overrides:
61             namespace: {resource: "namespace"}
62             pod: {resource: "pod"}
63         name:
64             matches: "^container_(.*)$"
65             as: ""
66     metricsQuery: 'sum(<<.Series>>{<<.LabelMatchers>>,container!="POD",
67         container!=""}) by (<<.GroupBy>>)'

```

Listing 4.13 Konfiguracja *Prometheus Adapter* — reguły mapowania (plik `k8s/monitoring/prometheus-adapter-config.yaml`)

4.5 Skrypty automatyzujące

4.5.1 deploy-k3s.sh — wdrożenie infrastruktury

Skrypt `scripts/deploy-k3s.sh` automatyzuje pełny proces wdrożenia:

1. **Budowanie obrazów Docker** — buduje obrazy dla `cpu-service`, `io-service` i `echo-server`.
2. **Import do containerd k3s** — importuje obrazy do *containerd* k3s przez `k3s ctr image import` (lub przez socket `containerd.sock`).
3. **Nakładanie manifestów** — tworzy namespace, wdraża monitoring (*Prometheus*, *Grafana*, *Prometheus Adapter*), serwisy aplikacyjne z HPA, `echo-server` i *Ingress*.
4. **Oczekiwanie na gotowość** — monitoruje rollout status dla każdego *Deploymentu* (timeout 120 s).

Skrypt obsługuje podkomendy: `deploy` (pełny proces), `build-only`, `apply-only`, `clean`, `status`.

4.5.2 run-tests.sh — orkiestracja testów

Skrypt `scripts/run-tests.sh` (~780 linii) automatyzuje wykonanie wszystkich 8 scenariuszy badawczych:

1. **Port-forward** — uruchamia `kubectl port-forward` dla `cpu-service` (`localhost:8080`) i `io-service` (`localhost:8081`), weryfikuje health check.
2. **Pętla scenariuszy** — dla każdego z 8 scenariuszy:
 - (a) Przełącza HPA na odpowiednią strategię (funkcja `switch_hpa`).
 - (b) Resetuje *Deployment* do 1 repliki (funkcja `reset_deployment`).
 - (c) Uruchamia test *k6* 5-krotnie (zmienna `REPETITIONS=5`).
 - (d) Zapisuje wyniki do `results/<scenario>/.`

3. **Cooldown** — po każdym scenariuszu: skaluje oba serwisy do 1 repliki, usuwa wszystkie HPA, czeka 120 sekund na stabilizację.

4. **Sprzątanie** — zatrzymuje port-forward.

```

1  -lzzlq--as-----znneq--alc--zlq-
2  # ---- lqPrzeczenie HPA
3  -----
4  # switch_hpa <service> <strategy>
5  #   service:  cpu-service / io-service
6  #   strategy: cpu / memory / custom / none
7  #
8  # Usuwa wszystkie istniejące HPA dla danego serwisu,
9  # następnie nakłada odpowiedni plik (chyba że strategy=none).
10 switch_hpa() {
11     local service="$1"
12     local strategy="$2"
13
14     info "lqPrzeczenie HPA dla ${service} → strategia: ${strategy}"
15
16     # Źuś wszystkie istniejące HPA dla tego serwisu
17     local hpa_names
18     hpa_names=$((${KUBECTL} -n "${NAMESPACE}" get hpa -o name 2>/dev/null |
19         grep "${service}" || true)
20     if [ -n "${hpa_names}" ]; then
21         for hpa in ${hpa_names}; do
22             ${KUBECTL} -n "${NAMESPACE}" delete "${hpa}" --ignore-not-found
23             =true
24         done
25         ok " Ę Usunito poprzednie HPA dla ${service}"
26     fi
27
28     # Nakadanie wybranej strategii
29     case "${strategy}" in
30         cpu)
31             ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-cpu.yaml"
32             ok " lż Naoono HPA CPU"
33             ;;
34         memory)
35             ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-memory.yaml"
36             ok " lż Naoono HPA Memory"
37             ;;
38         custom)
39             ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-custom.yaml"
40             ok " lż Naoono HPA Custom (RPS)"
41             ;;
42         none)
43             ok " Brak HPA (baseline)"
44             ;;
45         *)
46             err " Nieznana strategia: ${strategy}"
47             return 1
48             ;;
49     esac
50
51     sleep 2
52 }

```

Listing 4.14 Funkcja switch_hpa() — przełączanie strategii HPA (plik


```

16
17 // Scenario B - Image processing at constant load
18 process_image: {
19   executor: 'constant-vus',
20   vus: 10,
21   duration: '180s',
22   exec: 'processImage',
23   gracefulRampDown: '30s',
24 },
25 },
26
27 thresholds: {
28   // 95th percentile of request duration < 5 s
29   'http_req_duration': ['p(95)<5000'],
30   // Less than 1 % failed requests
31   'http_req_failed': ['rate<0.01'],
32 },
33 };

```

Listing 4.16 Scenariusze i progi w cpu-load-test.js

4.6.2 io-load-test.js

```

1 ---+---
2 export const options = {
3   scenarios: {
4     // Scenario A - Constant delay, ramp-up VUs
5     constant_query: {
6       executor: 'ramping-vus',
7       startVUs: 1,
8       stages: [
9         { duration: '120s', target: 100 }, // ramp-up
10        { duration: '180s', target: 100 }, // steady
11        { duration: '60s', target: 0 }, // ramp-down
12      ],
13       exec: 'constantQuery',
14       gracefulRampDown: '30s',
15     },
16
17     // Scenario B - Increasing delay (stress test)
18     stress_test: {
19       executor: 'constant-vus',
20       vus: 30,
21       duration: '120s',
22       exec: 'stressTest',
23       gracefulRampDown: '30s',
24     },
25
26     // Scenario C - External calls
27     external_calls: {
28       executor: 'constant-vus',
29       vus: 20,
30       duration: '120s',
31       exec: 'externalCalls',
32       gracefulRampDown: '30s',
33     },
34   },
35 };

```

```

36 thresholds: {
37   // 95th percentile of request duration < 10 s
38   'http_req_duration': ['p(95)<10000'],
39   // Less than 5 % failed requests
40   'http_req_failed': ['rate<0.05'],
41 },
42 };

```

Listing 4.17 Scenariusze i progi w io-load-test.js

4.6.3 combined-scenario.js

```

1  -----
2  export const options = {
3    scenarios: {
4      cpu_fibonacci: {
5        executor: 'ramping-vus',
6        startVUs: 0,
7        stages: [
8          { duration: '60s', target: 30 },
9          { duration: '180s', target: 30 },
10         { duration: '60s', target: 0 },
11       ],
12       exec: 'fibonacci',
13       gracefulRampDown: '30s',
14     },
15     cpu_processing: {
16       executor: 'constant-vus',
17       vus: 5,
18       duration: '300s',
19       exec: 'processImage',
20       startTime: '30s',
21       gracefulRampDown: '30s',
22     },
23     io_queries: {
24       executor: 'ramping-vus',
25       startVUs: 0,
26       stages: [
27         { duration: '60s', target: 80 },
28         { duration: '180s', target: 80 },
29         { duration: '60s', target: 0 },
30       ],
31       exec: 'ioQuery',
32       gracefulRampDown: '30s',
33     },
34   },
35
36   thresholds: {
37     'http_req_duration{scenario:cpu_fibonacci}': ['p(95)<5000'],
38     'http_req_duration{scenario:cpu_processing}': ['p(95)<10000'],
39     'http_req_duration{scenario:io_queries}': ['p(95)<10000'],
40     'http_req_failed': ['rate<0.05'],
41   },
42 };

```

Listing 4.18 Scenariusze i progi w combined-scenario.js

4.7 Podsumowanie implementacji

Zaimplementowany system badawczy składa się z czterech mikroservisów (`cpu-service`, `io-service`, `echo-server`, `memory-service`), infrastruktury monitorującej (*Prometheus*, *Grafana*, *Prometheus Adapter*) oraz zestawu skryptów automatyzujących wdrożenie i testy. Kluczowe cechy implementacji:

- **Pełna automatyzacja** — od budowy obrazów przez wdrożenie na k3s po wykonanie wszystkich scenariuszy testowych.
- **Powtarzalność** — każdy scenariusz testowy jest uruchamiany 5-krotnie, z cool-downem między scenariuszami.
- **Odtwarzalność** — wszystkie manifesty są deklaratywne i niezależne od konkretnego klastra *Kubernetes*.
- **Obserwowalność** — metryki zbierane przez *Prometheusa* i wizualizowane w *Grafanie* umożliwiają szczegółową analizę zachowania systemu pod obciążeniem.

Rozdział 5

Badania eksperymentalne i analiza wyników

Rozdział niniejszy opisuje metodykę badań eksperymentalnych oraz przedstawia analizę wyników. Badania zostały zaprojektowane w celu empirycznej weryfikacji hipotezy badawczej: skuteczność strategii HPA zależy od charakterystyki obciążeniowej aplikacji.

5.1 Metodyka badań

5.1.1 Środowisko testowe

Badania prowadzone są w dwóch środowiskach:

1. **Środowisko lokalne** — klaster *k3s* na serwerze QNAP TS-464 (8 GB RAM, 4-rdzeniowy procesor Intel Celeron). Wszystkie komponenty (mikroserwisy, monitoring, *prometheus-adapter*) działają w namespace **magisterka**. Ruch z *k6* do serwisów jest kierowany przez `kubectl port-forward`, co zapewnia minimalne opóźnienie sieciowe (< 1 ms).
2. **Środowisko chmurowe (GKE)** — zarządzany klaster *Google Kubernetes Engine* (GKE) na maszynach **e2-standard-4** (4 vCPU, 16 GB RAM). Środowisko chmurowe umożliwia weryfikację wyników uzyskanych lokalnie w warunkach zbliżonych do produkcyjnych.

5.1.2 Procedura eksperymentalna

Każdy z 8 scenariuszy badawczych (2 bazowe + 6 z macierzy 2×3) jest wykonywany według ściśle określonej procedury:

1. **Przygotowanie** — usunięcie wszystkich istniejących HPA dla docelowego serwisu, nałożenie pliku HPA odpowiadającego testowanej strategii.
2. **Reset** — skalowanie *Deploymentu* do 1 repliki (`kubectl scale --replicas=1`), oczekiwanie na gotowość (`rollout status`).
3. **Obciążenie** — uruchomienie skryptu *k6* odpowiedniego dla profilu obciążeniowego (`cpu-load-test.js` lub `io-load-test.js`).

4. **Zapis** — wyniki *k6* (JSON + tekstowe podsumowanie) oraz metadane uruchomienia zapisywane do `results/<scenario>/run-<i>/`.
5. **Powtórzenie** — kroki 1–4 powtarzane 5-krotnie dla każdego scenariusza.
6. **Stabilizacja** — po każdym scenariuszu: skalowanie obu serwisów do 1 repliki, usunięcie wszystkich HPA, oczekiwanie 120 sekund (*cooldown*).

5.1.3 Zbierane metryki

Dla każdego powtórzenia każdego scenariusza zbierane są następujące dane:

- **Metryki HTTP z k6**: liczba żądań, RPS (*throughput*), percentyle czasu odpowiedzi (p50, p95, p99), minimalny/maksymalny/średni czas odpowiedzi, liczba błędów HTTP (≥ 400).
- **Metryki infrastrukturalne z Prometheusa**: zużycie CPU per Pod (w milijednostkach), zużycie pamięci per Pod (w bajtach), liczba replik w czasie, stan HPA (pożądana vs. aktualna liczba replik).
- **Metadane**: scenariusz, strategia HPA, numer powtórzenia, timestamp, kod wyjścia k6.

5.1.4 Metryki badawcze — definicje operacyjne

Na podstawie surowych danych obliczane są następujące metryki badawcze:

1. **Średnia przepustowość (T)** — średnia liczba żądań na sekundę w trakcie testu, obliczana jako `total_requests/duration`.
2. **Opóźnienie p95 (L_{95})** — 95. percentyl czasu odpowiedzi HTTP (ms). Reprezentuje najgorszy akceptowalny czas odpowiedzi.
3. **Czas do skalowania (t_{scale})** — czas od momentu przekroczenia progu HPA do osiągnięcia docelowej liczby replik. Definiowany jako $t_{target_replicas} - t_{threshold_crossed}$.
4. **Efektywność skalowania (E)** — stosunek względnej zmiany przepustowości do względnej zmiany liczby replik:

$$E = \frac{\Delta T / T_{\min}}{\Delta R / R_{\min}} \quad (5.1)$$

gdzie $T_{\min}$ i $T_{\max}$ to minimalna i maksymalna przepustowość w trakcie testu, a $R_{\min}$ i $R_{\max}$ to minimalna i maksymalna liczba replik. Wartość $E = 1$ oznacza skalowanie liniowe; $E < 1$ oznacza malejące korzyści z dodawania replik; $E > 1$ oznacza ponadliniową poprawę (rzadkie).

5. **Stopa błędów (e)** — odsetek żądań zakończonych kodem HTTP ≥ 400 :

$$e = \frac{failed_requests}{total_requests} \times 100\% \quad (5.2)$$

6. **Amplituda skalowania** (A_R) — różnica między maksymalną a minimalną liczbą replik w trakcie testu: $A_R = R_{\max} - R_{\min}$. Większa amplituda oznacza, że HPA zareagował na obciążenie.
7. **Średnie zużycie CPU** — średnie zużycie procesora w milijednostkach (mCPU) na Pod, uśrednione po wszystkich Podach w trakcie testu.

5.2 Scenariusze testowe

5.2.1 Scenariusze bazowe (bez HPA)

Scenariusze `baseline-cpu` i `baseline-io` testują zachowanie systemu bez mechanizmu autoskalowania — liczba replik jest stała (1). Służą jako punkt odniesienia do oceny, czy HPA w ogóle poprawia wydajność.

Parametr	<code>baseline-cpu</code>	<code>baseline-io</code>
Serwis	<code>cpu-service</code>	<code>io-service</code>
HPA	brak	brak
Repliki (stałe)	1	1
Skrypt k6	<code>cpu-load-test.js</code>	<code>io-load-test.js</code>
Profil obciążenia	Fibonacci 1→50 VU + image processing 10 VU	stałe zapytania 1→100 VU + <code>stres-test</code> + <code>external</code>

Tabela 5.1 Parametry scenariuszy bazowych

5.2.2 Scenariusze CPU-bound

Parametr	<code>cpu-cpu</code>	<code>cpu-memory</code>	<code>cpu-custom</code>
Serwis	<code>cpu-service</code>	<code>cpu-service</code>	<code>cpu-service</code>
HPA	CPU 50%	Memory 70%	Custom RPS 100/Pod
<code>minReplicas</code>	2	2	2
<code>maxReplicas</code>	10	10	10
Skrypt k6	<code>cpu-load-test.js</code>	<code>cpu-load-test.js</code>	<code>cpu-load-test.js</code>

Tabela 5.2 Parametry scenariuszy CPU-bound

5.2.3 Scenariusze I/O-bound

Parametr	<code>io-cpu</code>	<code>io-memory</code>	<code>io-custom</code>
Serwis	<code>io-service</code>	<code>io-service</code>	<code>io-service</code>
HPA	CPU 50%	Memory 70%	Custom RPS 100/Pod
<code>minReplicas</code>	2	2	2
<code>maxReplicas</code>	10	10	10
Skrypt k6	<code>io-load-test.js</code>	<code>io-load-test.js</code>	<code>io-load-test.js</code>

Tabela 5.3 Parametry scenariuszy I/O-bound

5.3 Wyniki eksperymentalne — środowisko lokalne (k3s)

5.3.1 Scenariusze bazowe

Metryka	baseline-cpu	baseline-io
Średni RPS	TODO	TODO
p50 (ms)	TODO	TODO
p95 (ms)	TODO	TODO
p99 (ms)	TODO	TODO
Stopa błędów (%)	TODO	TODO
Średnie zużycie CPU (mCPU)	TODO	TODO
Średnie zużycie pamięci (MiB)	TODO	TODO

Tabela 5.4 Wyniki scenariuszy bazowych (średnia z 5 powtórzeń)

5.3.2 Scenariusze CPU-bound — porównanie strategii HPA

Metryka	cpu-cpu	cpu-memory	cpu-custom
Średni RPS	TODO	TODO	TODO
p50 (ms)	TODO	TODO	TODO
p95 (ms)	TODO	TODO	TODO
p99 (ms)	TODO	TODO	TODO
Stopa błędów (%)	TODO	TODO	TODO
Amplituda skalowania (A_R)	TODO	TODO	TODO
Czas do skalowania (s)	TODO	TODO	TODO
Efektywność skalowania (E)	TODO	TODO	TODO
Maks. liczba replik	TODO	TODO	TODO
Średnie zużycie CPU (mCPU)	TODO	TODO	TODO

Tabela 5.5 Wyniki scenariuszy CPU-bound (średnia z 5 powtórzeń)

5.3.3 Scenariusze I/O-bound — porównanie strategii HPA

Metryka	io-cpu	io-memory	io-custom
Średni RPS	TODO	TODO	TODO
p50 (ms)	TODO	TODO	TODO
p95 (ms)	TODO	TODO	TODO
p99 (ms)	TODO	TODO	TODO
Stopa błędów (%)	TODO	TODO	TODO
Amplituda skalowania (A_R)	TODO	TODO	TODO
Czas do skalowania (s)	TODO	TODO	TODO
Efektywność skalowania (E)	TODO	TODO	TODO
Maks. liczba replik	TODO	TODO	TODO
Średnie zużycie CPU (mCPU)	TODO	TODO	TODO

Tabela 5.6 Wyniki scenariuszy I/O-bound (średnia z 5 powtórzeń)

5.3.4 Wykresy przebiegu skalowania

Rysunek 5.1 Przebieg skalowania dla scenariusza `cpu-cpu` — liczba replik w czasie

Rysunek 5.2 Przebieg skalowania dla scenariusza `io-custom` — liczba replik w czasie

5.4 Analiza wyników

5.4.1 Porównanie strategii HPA dla CPU-bound

Oczekiwany rezultat: Dla obciążenia *CPU-bound*, strategia HPA CPU powinna być najskuteczniejsza, ponieważ zużycie CPU silnie koreluje z liczbą żądań. Strategia HPA Memory powinna być nieskuteczna (brak skalowania), ponieważ pamięć jest alokowana statycznie i nie zmienia się pod obciążeniem. Strategia HPA Custom RPS powinna być skuteczna, ponieważ RPS rośnie proporcjonalnie do obciążenia.

5.4.2 Porównanie strategii HPA dla I/O-bound

Oczekiwany rezultat: Dla obciążenia *I/O-bound*, strategia HPA CPU powinna być nieskuteczna (brak skalowania), ponieważ zużycie CPU nie koreluje z liczbą żądań — serwis spędza czas na `asyncio.sleep()`. Strategia HPA Memory również powinna być nieskuteczna. Strategia HPA Custom RPS powinna być najskuteczniejsza, ponieważ RPS rośnie proporcjonalnie do obciążenia niezależnie od profilu aplikacji.

5.4.3 Efektywność skalowania

Scenariusz	Efektywność E	Amplituda A_R	Czas skalowania (s)
cpu-cpu	TODO	TODO	TODO
cpu-memory	TODO	TODO	TODO
cpu-custom	TODO	TODO	TODO
io-cpu	TODO	TODO	TODO
io-memory	TODO	TODO	TODO
io-custom	TODO	TODO	TODO

Tabela 5.7 Efektywność skalowania — porównanie strategii

5.5 Wyniki eksperymentalne — środowisko chmurowe (GKE)

5.6 Dyskusja i interpretacja

Niniejsza sekcja będzie zawierać:

- Porównanie wyników lokalnych (k3s) i chmurowych (GKE).
- Identyfikację czynników wpływających na rozbieżności (opóźnienie sieciowe, różnice sprzętowe).
- Walidację lub falsyfikację hipotezy badawczej.
- Ograniczenia metodologii (np. pojedynczy węzeł k3s, sztuczny charakter obciążenia).

5.7 Wnioski z badań

Oczekiwane wnioski:

1. **HPA CPU** jest skuteczne wyłącznie dla aplikacji *CPU-bound*, gdzie zużycie procesora koreluje z obciążeniem. Dla aplikacji *I/O-bound* HPA CPU nie reaguje na wzrost ruchu.
2. **HPA Memory** jest nieskuteczne dla obu profili obciążeniowych — pamięć nie zmienia się proporcjonalnie do liczby żądań w testowanych aplikacjach.
3. **HPA Custom RPS** jest uniwersalnie skuteczne — niezależnie od profilu aplikacji, liczba żądań na sekundę koreluje z obciążeniem, co czyni tę strategię najbardziej odporną na różnice w charakterystyce aplikacji.
4. **Wybór metryki HPA musi być dopasowany do profilu aplikacji** — hipoteza badawcza zostaje potwierdzona.

Rozdział 6

Rozszerzenie *SaaS Multi-Tenancy*

6.1 Status implementacji

Zgodnie z planem pracy (Faza VIII), rozszerzenie systemu o model *SaaS Multi-Tenancy* zostało zaplanowane jako końcowy etap projektu. Na obecnym etapie prac (stan na czerwiec 2026) funkcjonalność *multi-tenancy* **nie została zaimplementowana** w kodzie źródłowym. W repozytorium nie występują nagłówki *X-Tenant-ID*, mechanizmy izolacji danych między tenantami ani konfiguracje *namespace-per-tenant*.

Niniejszy rozdział przedstawia zaprojektowaną koncepcję rozszerzenia *SaaS*, która stanowi podstawę do przyszłej implementacji.

6.2 Koncepcja *Multi-Tenancy* w systemie badawczym

6.2.1 Model izolacji tenantów

Zaprojektowano dwa poziomy izolacji tenantów — lekki (*application-level*) i pełny (*namespace-per-tenant*):

1. **Application-level multi-tenancy** — tenanty są rozróżniani na poziomie aplikacji przez nagłówek *X-Tenant-ID*. Wszyscy tenanty współdzielą ten sam *Deployment* i *namespace*. Izolacja dotyczy wyłącznie danych (np. klucze cache w *memory-service* są prefiksowane identyfikatorem tenanta), natomiast zasoby obliczeniowe są współdzielone. Ten poziom jest najprostszy w implementacji i najbardziej ekonomiczny, ale nie zapewnia izolacji wydajnościowej.
2. **Namespace-per-tenant** — każdy tenant otrzymuje dedykowany *namespace* z własnymi *Deploymentami*, *HPA*, *ResourceQuota* i *NetworkPolicy*. Zapewnia to pełną izolację zasobów i wydajności, ale zwiększa koszt infrastrukturalny proporcjonalnie do liczby tenantów.

6.2.2 Projektowana architektura

W modelu *application-level multi-tenancy*:

- **Nagłówek X-Tenant-ID** — każdy mikroserwis odczytuje identyfikator tenanta z nagłówka HTTP. W przypadku jego braku, żądanie jest odrzucane z kodem 400.

- **Izolacja danych** — *memory-service* używa prefiksów kluczy (`<tenant_id>:<key>`), zapewniając logiczną separację danych w obrębie jednego *in-memory cache*.
- **Metryki per tenant** — *Prometheus* zbiera metryki z etykietą `tenant_id`, umożliwiając analizę wydajności w podziale na tenantów.
- **HPA z etykietami** — *HorizontalPodAutoscaler* uwzględnia obciążenie od wszystkich tenantów łącznie, co odzwierciedla realistyczny scenariusz SaaS, gdzie infrastruktura jest współdzielona.

W modelu *namespace-per-tenant*:

- Każdy tenant ma własny *namespace* (np. `tenant-001`, `tenant-002`) z kompletnym zestawem zasobów.
- *ResourceQuota* ogranicza sumaryczne zużycie CPU i pamięci per tenant.
- *NetworkPolicy* izoluje ruch sieciowy między tenantami.
- Każdy tenant ma niezależny HPA, co umożliwia precyzyjne skalowanie per tenant.

6.3 Planowane scenariusze testowe

Po zaimplementowaniu rozszerzenia *multi-tenancy* planowane jest przeprowadzenie dodatkowych testów:

1. **Test izolacji wydajnościowej** — symulacja scenariusza „głośnego sąsiada” (*noisy neighbor*), w którym jeden tenant generuje wysokie obciążenie, a drugi wysła stały, niski ruch. Celem jest sprawdzenie, czy HPA chroni „cichego” tenanta przed degradacją wydajności.
2. **Test skalowania z tenantami** — porównanie efektywności HPA w wariancie z jednym tenantem vs. z wieloma tenantami. Oczekuje się, że HPA Custom RPS zachowa skuteczność niezależnie od liczby tenantów.
3. **Test kosztu izolacji** — porównanie zużycia zasobów w modelu *application-level* vs. *namespace-per-tenant* przy tej samej liczbie tenantów i poziomie obciążenia.

6.4 Oczekiwane wyzwania

- **Narzut nagłówka X-Tenant-ID** — każdy mikroserwis musi walidować i propagować nagłówki, co wymaga modyfikacji kodu wszystkich serwisów.
- **Eksplzja metryk** — metryki z etykietą `tenant_id` zwiększają kardynalność szeregów czasowych, co może przeciążyć *Prometheusa* przy dużej liczbie tenantów.
- **Ograniczenia sprzętowe** — QNAP TS-464 (8 GB RAM) może nie pomieścić wielu tenantów w modelu *namespace-per-tenant* ze względu na narzut *Kubernetes* na każdy *namespace*.
- **Synchronizacja HPA** — przy wielu tenantach współdzielących *Deployment*, HPA może „przeskalować” w odpowiedzi na jednego „głośnego” tenanta, niepotrzebnie zwiększając koszty.

6.5 Podsumowanie

Rozszerzenie *SaaS Multi-Tenancy* stanowi logiczną kontynuację badań nad skalowalnością — po ustaleniu, która strategia HPA jest najskuteczniejsza dla danego profilu obciążeniowego, kolejnym krokiem jest sprawdzenie, czy wnioski te utrzymują się w środowisku wielodostępnym (*multi-tenant*). Implementacja tego rozszerzenia jest zaplanowana jako Faza VIII projektu i nie została jeszcze zrealizowana.

Rozdział 7

Podsumowanie

7.1 Podsumowanie zrealizowanych prac

W ramach niniejszej pracy magisterskiej zrealizowano następujące cele:

1. **Analiza literatury** — przeprowadzono przegląd istniejących badań nad autoskalowaniem w *Kubernetes*, identyfikując cztery kluczowe luki badawcze: brak systematycznego porównania strategii HPA dla różnych profili obciążeniowych, ograniczoną analizę HPA Memory, brak badań łączących HPA z modelem *SaaS* oraz brak otwartego, odtwarzalnego środowiska badawczego.
2. **Zaprojektowanie architektury** — zaprojektowano system badawczy oparty na macierzy 2×3 (dwa profile obciążeniowe $\times$ trzy strategie HPA), składający się z czterech mikroservisów (`cpu-service`, `io-service`, `echo-server`, `memory-service`), infrastruktury monitorującej (*Prometheus*, *Grafana*, *Prometheus Adapter*) oraz zestawu skryptów automatyzujących.
3. **Implementacja** — zaimplementowano kompletny system badawczy w języku Python 3.12 z frameworkiem *FastAPI*, konteneryzowany za pomocą *Docker* i wdrażany na klastrze *k3s*. Wszystkie manifesty *Kubernetes* (Deploymenty, Serwisy, HPA, Ingress) oraz konfiguracje monitoringu są dostępne w repozytorium.
4. **Automatyzacja testów** — opracowano skrypt `run-tests.sh` orkiestrujący wykonanie 8 scenariuszy testowych z 5-krotnym powtórzeniem każdego, z automatycznym przełączaniem HPA, resetowaniem *Deploymentów* i cooldownem między scenariuszami.
5. **Scenariusze obciążeniowe** — zdefiniowano trzy skrypty *k6* (`cpu-load-test.js`, `io-load-test.js`, `combined-scenario.js`) z precyzyjnymi progami akceptacji i profilami obciążenia dopasowanymi do charakterystyki aplikacji.

7.2 Weryfikacja hipotezy badawczej

Hipoteza badawcza postawiona w Rozdziale 1 brzmiała:

„Nie istnieje jedna uniwersalna metryka HPA — skuteczność strategii skalowania zależy od charakterystyki obciążeniowej aplikacji (workload profile). Błędny wybór metryki prowadzi do niedoskalowania lub przeskalowania.”

Na obecnym etapie prac (czerwiec 2026) badania eksperymentalne nie zostały jeszcze przeprowadzone, w związku z czym hipoteza nie może zostać formalnie zweryfikowana. Poniżej przedstawiono oczekiwane rezultaty oparte na analizie teoretycznej i charakterystyce zaimplementowanych aplikacji:

- **HPA CPU** — oczekuje się, że będzie skuteczne wyłącznie dla `cpu-service`, gdzie zużycie procesora silnie koreluje z liczbą żądań (operacje *Pillow*, rekurencyjny Fibonacciego). Dla `io-service`, gdzie procesor jest bezczynny podczas `asyncio.sleep()`, HPA CPU nie powinno reagować na wzrost ruchu.
- **HPA Memory** — oczekuje się, że będzie nieskuteczne dla obu serwisów, ponieważ ani `cpu-service`, ani `io-service` nie alokują pamięci dynamicznie w odpowiedzi na obciążenie. Pamięć jest alokowana statycznie przy starcie i utrzymuje stały poziom.
- **HPA Custom RPS** — oczekuje się, że będzie uniwersalnie skuteczne, ponieważ liczba żądań na sekundę rośnie proporcjonalnie do obciążenia niezależnie od tego, czy żądania są CPU-intensywne, czy I/O-intensywne.

7.3 Ograniczenia pracy

Należy wskazać następujące ograniczenia niniejszej pracy:

- **Brak wyników eksperymentalnych** — na obecnym etapie zaimplementowano kompletny system badawczy i narzędzia do testów, ale same badania nie zostały jeszcze przeprowadzone. Wnioski przedstawione w Rozdziale 5 mają charakter oczekiwany i wymagają walidacji empirycznej.
- **Pojedynczy węzeł k3s** — lokalne środowisko testowe działa na pojedynczym serwerze QNAP, co oznacza, że wszystkie Pody są uruchamiane na tym samym węźle. W środowisku wielowęzłowym efekty związane z rozkładem obciążenia między węzłami mogłyby wpłynąć na wyniki.
- **Sztuczny charakter obciążenia** — aplikacje badawcze (`cpu-service`, `io-service`) symulują określone profile obciążeniowe, ale nie odzwierciedlają rzeczywistych aplikacji produkcyjnych. Rzeczywiste systemy często wykazują mieszane profile (*CPU-bound* + *I/O-bound*).
- **Ograniczony zakres HPA** — badanie ogranicza się do trzech metryk HPA (*CPU*, *Memory*, *Custom RPS*). Nie analizowano innych strategii, takich jak HPA na podstawie liczby żądań w toku (*in-flight*), metryk zewnętrznych (*External Metrics*) ani skalowania predykcyjnego.
- **Brak implementacji *multi-tenancy*** — rozszerzenie *SaaS Multi-Tenancy* (Faza VIII) zostało zaprojektowane koncepcyjnie, ale nie zaimplementowane w kodzie. Wnioski dotyczące zachowania systemu w środowisku wielodostępnym pozostają hipotetyczne.
- **Ograniczenia sprzętowe** — serwer QNAP TS-464 z 8 GB RAM ogranicza skalę eksperymentów. Przy `maxReplicas=10` dla obu serwisów plus monitoring, dostępna pamięć może być niewystarczająca.

7.4 Rekomendacje praktyczne

Na podstawie analizy teoretycznej i projektu systemu sformułowano następujące rekomendacje dla praktyków:

1. **Dobieraj metrykę HPA do profilu aplikacji** — przed wyborem strategii autoskalowania należy scharakteryzować aplikację pod kątem profilu obciążeniowego (*CPU-bound*, *I/O-bound*, *memory-bound*). Domyślna strategia HPA CPU może być nieskuteczna dla aplikacji I/O-bound.
2. **Używaj HPA Custom RPS jako strategii uniwersalnej** — jeśli charakterystyka aplikacji jest nieznana lub zmienna, metryka RPS (dostarczana przez *prometheus-adapter*) jest bezpiecznym wyborem, ponieważ koreluje z obciążeniem niezależnie od profilu.
3. **Unikaj HPA Memory jako jedynej metryki** — chyba że aplikacja wykazuje silną korelację między obciążeniem a zużyciem pamięci (np. cache, kolejki), HPA Memory prawdopodobnie nie będzie skuteczne.
4. **Testuj w środowisku odzwierciedlającym produkcję** — wnioski z testów na pojedynczym węźle k3s niekoniecznie przenoszą się na środowisko wielowęzłowe lub chmurowe. Rekomenduje się walidację wyników na GKE lub innym klastrze produkcyjnym.
5. **Automatyzuj testy porównawcze** — ręczne przełączanie HPA i uruchamianie testów jest podatne na błędy. Skrypt `run-tests.sh` opracowany w ramach niniejszej pracy może służyć jako wzorzec do automatyzacji podobnych badań.

7.5 Dalsze kierunki badań

Niniejsza praca otwiera następujące kierunki dalszych badań:

1. **Przeprowadzenie badań eksperymentalnych** — najpilniejszym kolejnym krokiem jest wykonanie 8 scenariuszy testowych na klastrze k3s i zebranie danych empirycznych do weryfikacji hipotezy.
2. **Wdrożenie na GKE** — powtórzenie eksperymentów na zarządzanym klastrze *Google Kubernetes Engine* w celu zweryfikowania, czy wyniki uzyskane lokalnie są odtwarzalne w środowisku chmurowym.
3. **Implementacja *multi-tenancy*** — zrealizowanie Fazy VIII zgodnie z koncepcją opisaną w Rozdziale 6 i przeprowadzenie testów w środowisku wielodostępnym.
4. **Rozszerzenie macierzy badawczej** — dodanie `memory-service` jako trzeciego profilu obciążeniowego (*memory-bound*) i zbadanie, czy HPA Memory staje się skuteczne dla aplikacji, których zużycie pamięci koreluje z obciążeniem.
5. **Badanie innych metryk HPA** — analiza skuteczności metryk takich jak liczba żądań w toku (`http_requests_inflight`), metryk zewnętrznych (długość kolejki *RabbitMQ*, liczba wiadomości *Kafka*) oraz kombinacji wielu metryk w jednym HPA.

6. **Skalowanie predykcyjne** — porównanie reaktywnego HPA z podejściami predykcyjnymi (np. opartymi na LSTM), które przewidują przyszłe obciążenie i skalują proaktywnie.
7. **Analiza kosztowa** — rozszerzenie badań o analizę kosztów infrastruktury w modelu *pay-as-you-go* dla różnych strategii HPA, szczególnie w kontekście *multi-tenancy*, gdzie przeskalowanie przekłada się bezpośrednio na wyższe koszty.
8. **Testy z rzeczywistymi aplikacjami** — zastąpienie syntetycznych serwisów badawczych rzeczywistymi aplikacjami (np. system rezerwacji, przetwarzanie obrazu w pipeline ML) w celu walidacji uniwersalności wniosków.

7.6 Podsumowanie końcowe

Niniejsza praca magisterska stanowi kompleksowe przygotowanie do badań empirycznych nad skutecznością strategii *HorizontalPodAutoscaler* w *Kubernetes* dla różnych profili obciążeniowych. Zaimplementowano w pełni funkcjonalny system badawczy składający się z mikroservisów, infrastruktury monitorującej i skryptów automatyzujących, gotowy do przeprowadzenia eksperymentów.

Główny wkład pracy obejmuje:

- Systematyczną analizę literatury identyfikującą luki badawcze w dziedzinie auto-skalowania w *Kubernetes*.
- Projekt macierzy badawczej 2×3 umożliwiającej izolowane porównanie strategii HPA.
- Implementację czterech mikroservisów o kontrolowanych profilach obciążeniowych (*CPU-bound*, *I/O-bound*, *memory-bound*).
- Konfigurację infrastruktury monitorującej z niestandardowymi metrykami przez *prometheus-adapter*.
- Pełną automatyzację procesu badawczego — od wdrożenia po wykonanie testów.
- Otwarte, odtwarzalne środowisko badawcze dostępne w repozytorium.

Przeprowadzenie zaplanowanych badań eksperymentalnych i weryfikacja hipotezy badawczej stanowią bezpośredni kolejny krok, do którego niniejsza praca dostarcza wszystkich niezbędnych narzędzi.

Dodatek A

Spis listingów

Bibliografia

- [1] E. Casalicchio, V. Perciballi. Auto-scaling of containers: The impact of relative and absolute metrics. *2021 IEEE 5th International Workshop on Smart Container Technologies and Orchestration*, strony 1–6. IEEE, 2021.
- [2] T.-T. Nguyen, Y.-J. Kim. A proactive autoscaling approach using lstm-based traffic prediction for kubernetes. *IEEE Access*, 8:149486–149500, 2020.

Spis rysunków

- 5.1 Przebieg skalowania dla scenariusza `cpu-cpu` — liczba replik w czasie . . . 45
- 5.2 Przebieg skalowania dla scenariusza `io-custom` — liczba replik w czasie . 45

Spis tabel

2.1	Porównanie <i>k3s</i> i standardowego <i>Kubernetes</i>	8
2.2	Porównanie środowiska lokalnego (<i>k3s</i>) i chmurowego (<i>GKE</i>)	12
2.3	Podsumowanie stosu technologicznego projektu	13
3.1	Macierz badawcza 2×3 — identyfikatory scenariuszy testowych	15
3.2	Zasoby przydzielone serwisom (<i>resource requests</i> i <i>limits</i>)	20
5.1	Parametry scenariuszy bazowych	43
5.2	Parametry scenariuszy CPU-bound	43
5.3	Parametry scenariuszy I/O-bound	43
5.4	Wyniki scenariuszy bazowych (średnia z 5 powtórzeń)	44
5.5	Wyniki scenariuszy CPU-bound (średnia z 5 powtórzeń)	44
5.6	Wyniki scenariuszy I/O-bound (średnia z 5 powtórzeń)	45
5.7	Efektywność skalowania — porównanie strategii	46